

AN 978: Nios[®] V Processor Migration Guidelines

Contents

1. Overview.....	3
2. Architecture Comparison.....	4
2.1. Features Comparison.....	4
2.2. Ecosystem Comparison.....	6
2.3. Software Optimization.....	6
3. Processor Core Upgrade Paths.....	8
4. Design Flow Comparison.....	9
4.1. Differences in the Design Flow.....	9
4.2. Design Upgrade from Nios II Processor to Nios V Processor.....	10
4.2.1. Quartus Prime Standard Edition.....	11
4.2.2. Quartus Prime Pro Edition.....	16
5. Processor Core Comparison.....	23
5.1. Upgrading Nios II/e Processor to Nios V/c Processor	23
5.1.1. Hardware Perspective.....	23
5.1.2. Software Perspective.....	27
5.2. Upgrading Nios II Processor to Nios V/m Processor.....	29
5.2.1. Hardware Perspective.....	29
5.2.2. Software Perspective.....	36
5.3. Upgrading Nios II/f Processor to Nios V/g Processor.....	38
5.3.1. Hardware Perspective.....	38
5.3.2. Software Perspective.....	49
6. Feature Comparison.....	52
6.1. Timer.....	52
6.2. Interrupt.....	53
6.3. Shadow Register.....	54
6.4. Ethernet Stack.....	55
6.5. Bootloader.....	56
6.6. Data and Instruction Cache.....	57
6.7. Tightly Coupled Memory.....	58
6.8. Custom Instructions.....	59
6.9. Error Correction Code.....	60
6.10. Altera HAL Settings.....	61
6.11. Micrium MicroC/OS-II BSP Settings.....	63
6.12. Software Packages.....	65
7. Assembly Language Comparison.....	66
7.1. Arithmetic and Logic Operations.....	67
7.2. Data Handling and Memory Operations.....	70
7.3. Control Flow Operations.....	72
7.4. Miscellaneous Operations.....	73
8. Document Revision History for the AN 978: Nios V Processor Migration Guidelines.....	74



1. Overview

This document describes the guidelines for upgrading from the Nios® II processor to the corresponding Nios V processor core.

Nios V processor offers the following benefits:

- RISC-V architecture, which has a wide ecosystem of support in open community, common tooling, and a broad variety of software vendor choices.
- Robust 32 bit ISA.
- A wide range of operating systems. Altera enables the Nios V processor to support Zephyr*, FreeRTOS*, and MicroC/OS-II Real-Time Operating System (RTOS).
- Supports standard native tooling, removing the need for Windows Subsystem for Linux (WSL) or Cygwin.
- Integrated development environment with Ashling* RiscFree* IDE for Altera® FPGAs that supports RISC-V specification and Altera Hard Processor System (HPS) software build and debug.

Related Information

- [Nios V Processor Reference Manual](#)
- [Nios V Embedded Processor Design Handbook](#)
- [Ashling* RiscFree* Integrated Development Environment \(IDE\) for Altera FPGAs User Guide](#)
- [Nios V Processor Software Developer Handbook](#)
- [AN 980: Nios V Processor Quartus® Prime Software Support](#)
- [AN 985: Nios V Processor Tutorial](#)
- [AN 977: Nios V Processor Custom Instruction](#)
- [Nios V Processor Altera FPGA IP Release Notes](#)

2. Architecture Comparison

2.1. Features Comparison

Table 1. Core Architecture Comparison to Nios II/e Processor

Feature	Core Architecture Comparison		
	Nios V/c Processor	Non-pipelined Nios V/m Processor	Nios II/e Processor
32-bit General Purpose Register (GPR)	Yes	Yes	Yes
Pipelined Architecture	—	—	—
Debug Module	—	Yes	Yes
M Extn (Mul/Div)	—	—	—
Custom Instructions	—	—	Yes
Cache Support	—	—	—
Tightly Coupled Memory (TCM)	—	—	—
Branch Prediction	—	—	—
On-Chip Trace	—	—	—
Error Correction Code (ECC)	Yes	Yes	—
32-bit Floating Point Unit (FPU)	—	—	—
Memory Protection Unit (MPU)	—	—	—
Memory Management Unit (MMU)	—	—	—
Internal Timer	—	Yes	—
Machine Mode	Yes	Yes	Yes
User Mode	—	—	—
Supervisor Mode	—	—	—
Interrupt Controller	—	Yes	Yes
Exception Handling	—	Yes	Yes
Altera HAL	Yes	Yes	Yes
uC-OS/II	—	Yes	Yes
Zephyr	—	Yes	—
FreeRTOS	—	Yes	—
Embedded Linux	—	—	—

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera and Intel warrant performance of its FPGA and semiconductor products to current specifications in accordance with Altera's or Intel's standard warranty as applicable, but reserves the right to make changes to any products and services at any time without notice. Altera and Intel assume no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera or Intel. Altera and Intel customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.

*Other names and brands may be claimed as the property of others.

**ISO
9001:2015
Registered**

Table 2. Core Architecture Comparison to Nios II/f Processor

Feature	Core Architecture Comparison		
	Nios V/m Processor	Nios V/g Processor	Nios II/f Processor
32-bit General Purpose Register (GPR)	Yes	Yes	Yes
Pipelined Architecture	Yes	Yes	Yes
Debug Module	Yes	Yes	Yes
M Extn (Mul/Div)	—	Yes	Yes
Custom Instructions	—	Yes	Yes
Cache Support	—	Yes	Yes
Tightly Coupled Memory (TCM)	—	Yes	Yes
Branch Prediction	—	Yes	Yes
On-Chip Trace	—	—	Yes
Error Correction Code (ECC)	Yes	Yes	Yes
32-bit Floating Point Unit (FPU)	—	Yes (F Extn)	Yes (FPH1 or FPH2)
Memory Protection Unit (MPU)	—	—	Yes
Memory Management Unit (MMU)	—	—	Yes
Internal Timer	Yes	Yes	—
Machine Mode	Yes	Yes	Yes
User Mode	—	—	—
Supervisor Mode	—	—	Yes
Interrupt Controller	Yes	Yes	Yes
Exception Handling	Yes	Yes	Yes
Intel HAL	Yes	Yes	Yes
uC-OS/II	Yes	Yes	Yes
Zephyr	Yes	Yes	—
FreeRTOS	Yes	Yes	—
Embedded Linux	—	—	Yes

2.2. Ecosystem Comparison

Table 3. Integrated Development Environment (IDE) and Software Ecosystem Comparison

Features	Ashling Visual Studio Code Extension for Altera FPGAs	Ashling RiscFree IDE for Altera FPGAs	Nios II Embedded Design Suite
IDE type	Visual Studio Code based	Eclipse based	Eclipse based
Toolchain	GNU RISC-V Embedded GCC Toolchain	GNU RISC-V Embedded GCC Toolchain	GNU GCC Toolchain
Automated scan chain	Yes	Yes	Yes
Core support	Nios V processor (Risc-V), ARM Cortex-A9, A53	Nios V processor (Risc-V), ARM Cortex-A9, A53	Nios II only
Heterogeneous debug	Yes	Yes	No
Trace ⁽¹⁾	No	On-chip Trace: ARM Cortex-A9, A53	No
Baremetal debug	Yes	Yes	Yes
uC/OS-II debug	No	Yes (With OS awareness)	Yes (Without OS awareness)
FreeRTOS debug	Yes	Yes (With OS awareness)	No
Zephyr debug	Yes	Yes (With OS awareness)	No

2.3. Software Optimization

The following information represents a Hello World application's program size with various software optimizations. You can evaluate your application ELF file using the `nios2-stack-report` and `niosv-stack-report` commands.

Both Nios II and Nios V processor software support the default Newlib C library and the option for Reduced Device Drivers. The difference lies in the optimized C library, which is the Small-C library in the Nios II processor, and the Picolibc library in the Nios V processor.

Example 1. Hello World Application

```
#include <stdio.h>
int main()
{
    printf("Hello from Nios II!\n");
    return 0;
}
```

Table 4. ELF Footprint Optimization Comparison

Board Support Package Settings	hal.make.cflags_optimization	Program Size (KB)	
		Nios II Processor	Nios V Processor
Default (Newlib)	O0	28	33
	O2	24	28
continued...			

⁽¹⁾ Nios V processor does not support trace feature.

Board Support Package Settings	hal.make.cflags_optimization	Program Size (KB)	
		Nios II Processor	Nios V Processor
	Os	23	27
Reduced Device Drivers	O0	20	26
	O2	17	22
	Os	17	22
Small-C	O0	14	—
	O2	10	—
	Os	10	—
Picolibc	O0	—	16
	O2	—	12
	Os	—	11
• Small-C • Reduced Device Drivers	O0	6	—
	O2	5	—
	Os	4	—
• Picolibc • Reduced Device Drivers	O0	—	9
	O2	—	6
	Os	—	6

Related Information

[Nios V Processor Software Developer Handbook](#)

For more information about reducing code footprint in embedded systems.



3. Processor Core Upgrade Paths

Table 5. Processor Core Upgrade Paths

Nios II Processor Variant	Nios V Processor Variant
Nios II/e processor (Without JTAG debug, interrupts, and exceptions)	Nios V/c processor
Nios II/e processor	Non-pipelined Nios V/m processor
- Nios II/e processor - Nios II/f processor (Basic features without multiply and divide units, cache, and custom instruction)	Nios V/m processor
Nios II/f processor (With multiply and divide units, cache, and custom instruction)	Nios V/g processor

Related Information

[Nios V Processor Altera FPGA IP Release Notes](#)

4. Design Flow Comparison

4.1. Differences in the Design Flow

Table 6. Notable Consideration by Design Flow

Design Stage	Nios V Processor	Nios II Processor	Notes
Quartus® Prime Project Creation	Create a new project using the New Project Wizard .	Create a new project using the New Project Wizard .	None
Define and Generate System in the Platform Designer	1. Instantiate the Nios V processor core. 2. Create a Nios V processor system with basic peripherals.	1. Instantiate the Nios II processor core. 2. Create a Nios II processor system with basic peripherals.	1. Nios V processor has a similar interface as the Nios II processor. You can replace the interface in Platform Designer. 2. Refer to Table <i>Processor Core Upgrade Paths</i> for the processor core mapping guidelines. 3. Refer to Table <i>Primary Interface</i> for the signals connection mapping guidelines.
Invoke the BSP Editor	• For Quartus Prime Pro Edition software: — Use Platform Designer. • For Quartus Prime Standard Edition software: — Use Nios V Command Shell with the command <code>niosv-bsp-editor</code>.	• You can use the Nios II Embedded Design Suite (Eclipse IDE) in both Quartus Prime software editions. • Alternatively, you can use the following software: — For Quartus Prime Pro Edition software, use Platform Designer. — For Quartus Prime Standard Edition software, use Nios II Command shell with the command <code>nios2-bsp-editor</code>.	Invoke the BSP Editor with a different tool.
Create system file for BSP Editor	• For Quartus Prime Pro Edition software, use <code>.qsys</code> file. • For Quartus Prime Standard Edition software, use <code>sopcinfo</code> file.	For both Quartus Prime software editions, use <code>sopcinfo</code> file.	Different input file to create the BSP settings file.
continued...			

Design Stage	Nios V Processor	Nios II Processor	Notes
Configure and generate the BSP	1. Decide what features the BSP requires by specifying the components in the BSP and relevant settings. 2. Generate the settings.bsp file using the following tools: • Option 1: BSP Editor in Platform Designer. • Option 2: niosv-bsp utility. 3. Import the BSP folder into Ashling RiscFree IDE for Altera FPGAs.	1. Decide what features the BSP requires by specifying the components in the BSP and relevant settings. 2. Generate the settings.bsp file using the following tools: • Option 1: BSP Editor in Nios II EDS • Option 2: nios2-bsp utility.	• Ashling RiscFree IDE for Altera FPGAs does not support new Nios V processor BSP project creation. You need to import the BSP project. • Apply different CLI command for BSP generation.
Generate the APP folder	1. Develop the application code based on the hardware design. 2. Generate an application build CMakeLists.txt using the niosv-app utility. 3. Import the APP folder into Ashling RiscFree IDE for Altera FPGAs.	1. Develop the application code based on the hardware design. 2. Generate an application build MakeFiles using the following tools: • Option 1: Nios II EDS • Option 2: nios2-app-generate-makefile utility.	• Ashling RiscFree IDE for Altera FPGAs does not support new Nios V processor APP project creation. You need to import the APP project. • Apply different CLI command for APP generation.
Build the ELF	Use one of the following tools: • Option 1: Ashling RiscFree IDE for Altera FPGAs. • Option 2: cmake and make command.	Use one of the following tools: • Option 1: Nios II EDS • Option 2: make command	Different development tools build flow.
Download the ELF	Use one of the following tools: • Option 1: Ashling RiscFree IDE for Altera FPGAs. • Option 2: niosv-download command.	Use one of the following tools: • Option 1: Nios II EDS • Option 2: nios2-download command.	Different development tools and ELF download command.
Open the JTAG UART Terminal	Use the <code>juart-terminal</code> command.	Use one of the following tools: • Option 1: Nios II Console in Nios II EDS • Option 2: nios2-terminal command.	1. Ashling RiscFree IDE for Altera FPGAs does not support native JTAG UART terminal. You can open the terminal using External Tools Configuration feature. 2. Different open terminal command for JTAG UART Altera FPGA IP.

Related Information

- [Nios V Embedded Processor Design Handbook](#)
- [Nios V Processor Reference Manual – Processor Architecture](#)
For more information about Nios V Processor Performance Benchmarks.

4.2. Design Upgrade from Nios II Processor to Nios V Processor

To upgrade a design from the Nios II processor to the Nios V processor, you must upgrade two basic components of the embedded system:

- Hardware component (Quartus Prime project)
- Software component (Eclipse IDE project)

The following simple Hello World design of the Nios II processor with on-chip memory and JTAG UART demonstrates the process of upgrading to Nios V processor.

Related Information

[Processor Core Comparison](#) on page 23

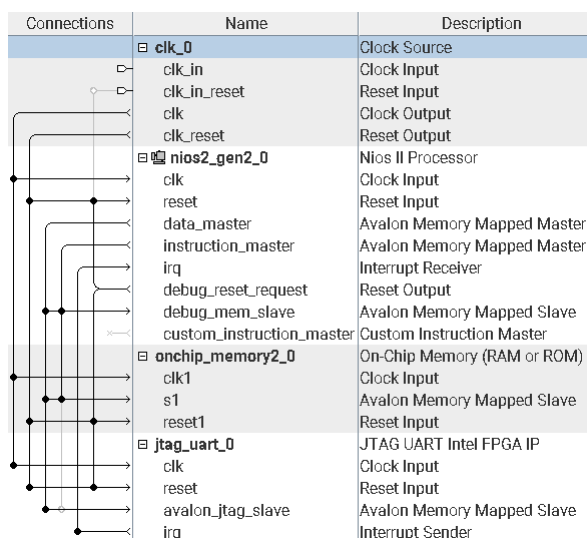
For more information about feature upgrade.

4.2.1. Quartus Prime Standard Edition

4.2.1.1. Hardware Component

If you compile your design using Quartus Prime Standard Edition, you can start upgrading to Nios V processor from Platform Designer. The following figure shows an example of the Nios II Processor QSYS design.

Figure 1. Nios II Processor Qsys Design



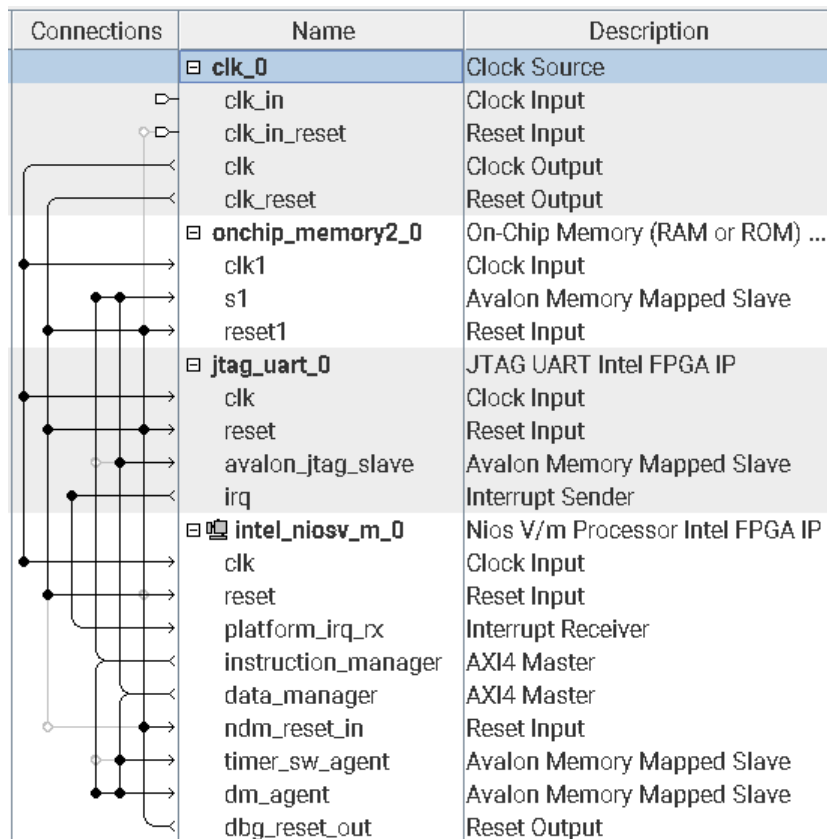
1. From IP Catalog, search for Nios V processor. Identify your preferred processor core.
2. Double-click on your selected processor.
3. The parameters window appears. Select **Enable Reset from Debug Module**. Click **Finish**.
4. The selected processor core is added to your system. Proceed to set the required connections based on the table below:

Table 7. Nios II Processor to Nios V Processor Interfaces

Interface	Nios V Processor	Nios II Processor	Description
Clock	clock	clock	Similar clock input
Reset	reset	reset	Similar hard reset
Debug reset	- dbg_reset_out - ndm_reset_in	debug_reset_request	- Mainly used to issue debug reset from CLI or IDE. - By default, this feature is enabled in Nios II/e processor but disabled in Nios V/m processor.
Interrupts	platform_irq_rx	irq	- Nios II/e processor supports 32 platform interrupts. - Nios V/m processor supports 16 platform interrupts.
Timer and Software Interrupt Module	timer_sw_agent	—	- Only supported in Nios V/m and Nios V/g processors. - By default, it is connected to the data manager.
Debug	dm_agent	debug_mem_slave	- Allow JTAG download and debug. For Nios V/m and Nios V/g processors, it is connected to the data manager and instruction manager by default. - Not applicable to Nios V/c processor.
Data bus	data_manager	data_master	- Nios II/e processor uses Avalon® memory-mapped interface. - Nios V/m processor uses Arm* AMBA* AXI-4.
Instruction bus	instruction_manager	instruction_master	- Nios II/e processor uses Avalon memory-mapped interface. - Nios V/m processor uses Arm AMBA AXI-4.

- Open your selected Nios V processor IP. Click **Vectors ► Reset Agent** and update the parameter as onchip_memory2_0.s1
- Remove Nios II processor from Platform Designer. A preview of the final system is shown as below.

Figure 2. Nios V Processor System

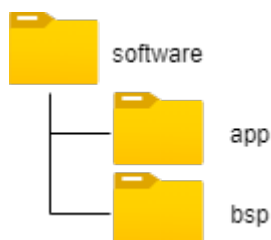


You can now generate the system HDL and compile Quartus Prime project. Use the Quartus Prime Programmer software to configure the design SOF file for design verification.

4.2.1.2. Software Component

The software development flow starts with the creation of board support package (BSP) and application (APP). Altera recommends creating a software folder with app and bsp sub-folders inside your Quartus Prime project directory. Nios II processor software also has the same file organization.

Figure 3. Nios V Processor Software Project File Directory

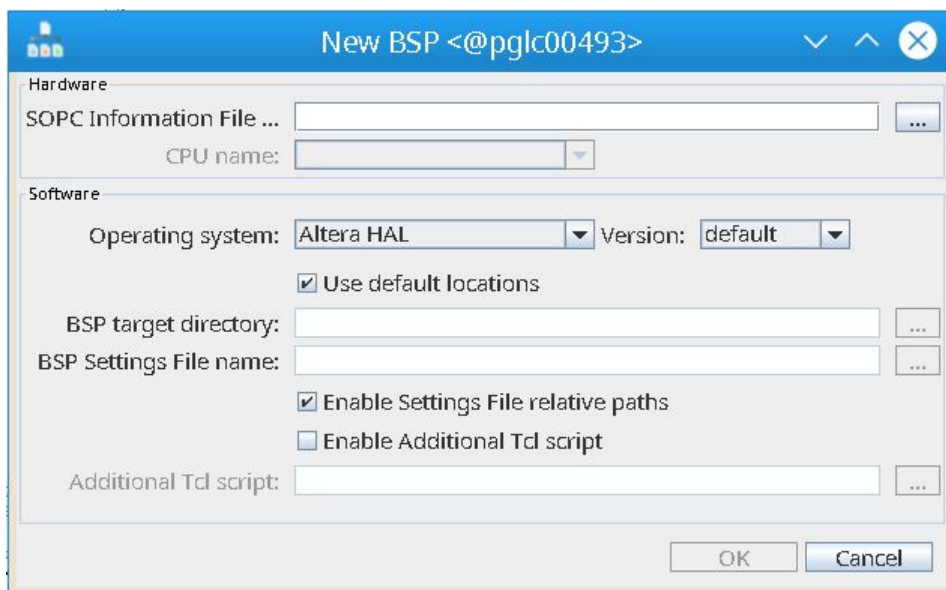


Note: Generated Nios II processor software files are not compatible with the Nios V processor. You need to regenerate the Nios V processor BSP and APP project.

Nios V Processor BSP Regeneration

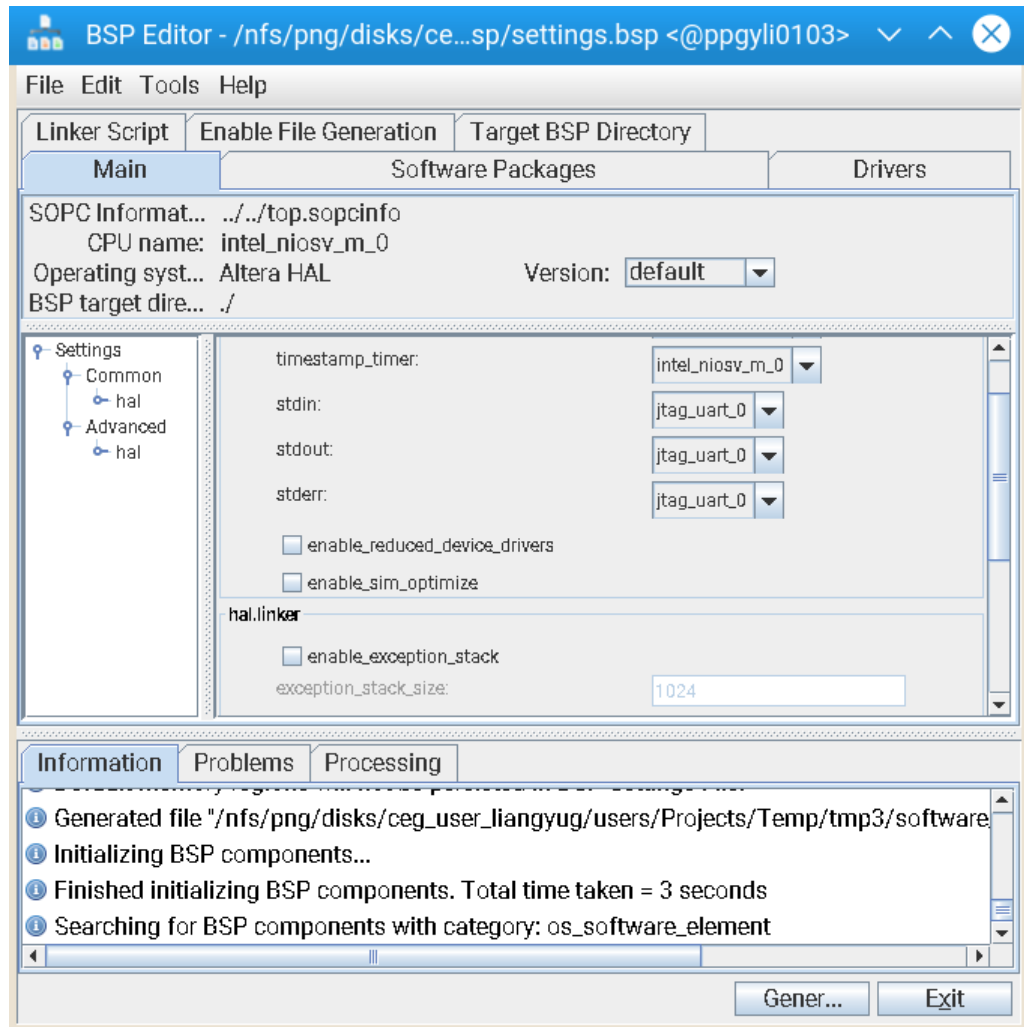
1. Open the **BSP Editor** in Nios V Command Shell with the command `niosv-bsp-editor`.
2. Select the design SOPCINFO file in the **SOPC Information File** field, then click **OK**.
3. The window assigns **BSP target directory** and **BSP Settings File name** to default locations.

Figure 4. New BSP Window



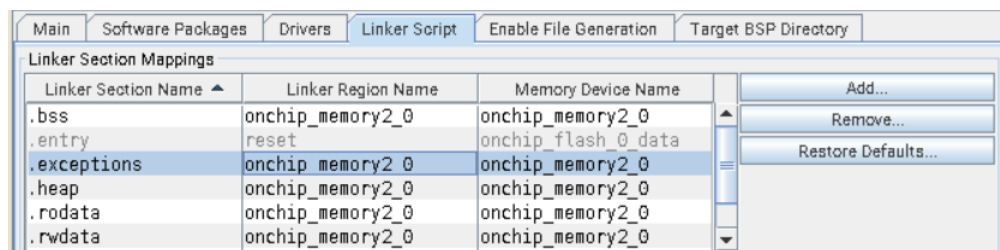
4. Apply the same Nios II processor BSP settings into the Nios V processor BSP settings. Refer to [Altera HAL Settings](#) to replicate the BSP Settings.

Figure 5. Nios V Processor BSP Settings



5. Select the appropriate memory region for exception vector in **BSP Editor** ➤ **Linker Script**.

Figure 6. Linker Script



6. Click **Generate BSP** to create a new BSP project for Nios V processor.

Nios V Processor APP Regeneration

1. Copy the Nios II application source codes into the APP folder.
2. Start the Nios V Command Shell.
3. Run `niosv-app` command to create an APP `CMakeLists.txt` file.

```
niosv-app -b=<BSP directory> -a=<APP directory> -s=<Nios II application source codes>
```

4. Run CMake executable.

```
cmake -B <APP directory>/build -S <APP directory> -G "Unix Makefiles"
```

5. Run Make executable. After successful Make, the Nios V processor software ELF file is located in the `<APP directory>/build` folder.

```
make -C <APP directory>/build
```

6. Download the software ELF file into the Nios V processor system.

```
niosv-download <Nios V ELF file>
```

7. Open JTAG UART terminal to print the Hello World message.

```
juart-terminal
```

Alternatively, you can use Ashling RiscFree IDE for Altera FPGAs for Nios V processor software development. After the BSP creation and APP creation, you can import both folders into the workspace of Ashling RiscFree IDE for Altera FPGAs to continue.

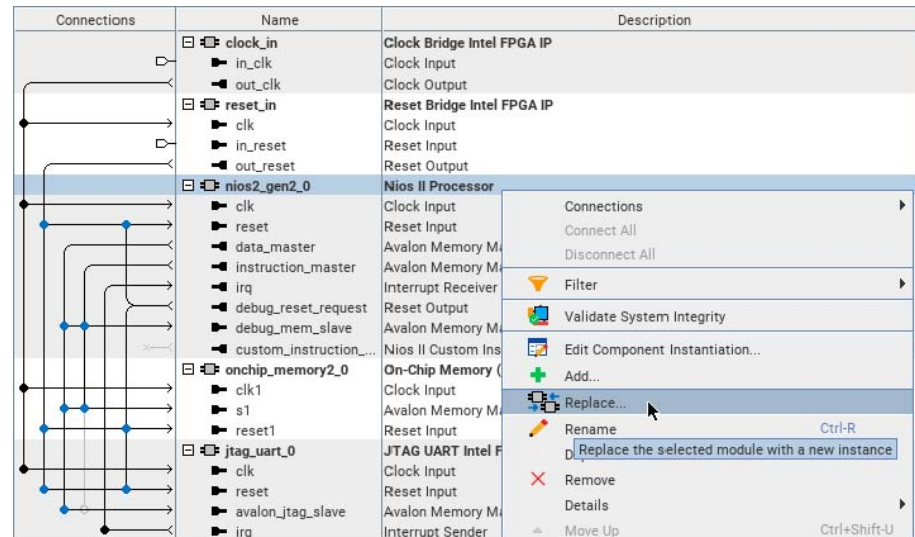
4.2.2. Quartus Prime Pro Edition

4.2.2.1. Hardware Component

If you compile your design using Quartus Prime Pro Edition, you can start upgrading from Nios V processor from Platform Designer. The following figure shows an example of the Nios II Processor QSYS design.

1. Right-click **Nios II Processor** in the **Description** column, and select **Replace**.

Figure 7. Nios II Processor Qsys Design



2. Search for Nios V processor from the **Replace** instance. Select your preferred processor core.
3. Double-click on your selected Nios V processor IP. The parameters window appears.
4. Select **Enable Reset from Debug Module** and click **Finish**.
5. In the **Connection Migration Tool > Interface mapper**, reconnect:
 - data_master to data_manager
 - instruction_master to instruction_manager
 - debug_mem_slave to dm_agent
6. Click **OK**. Nios V processor replaces Nios II processor.

Figure 8. Interface Mapper

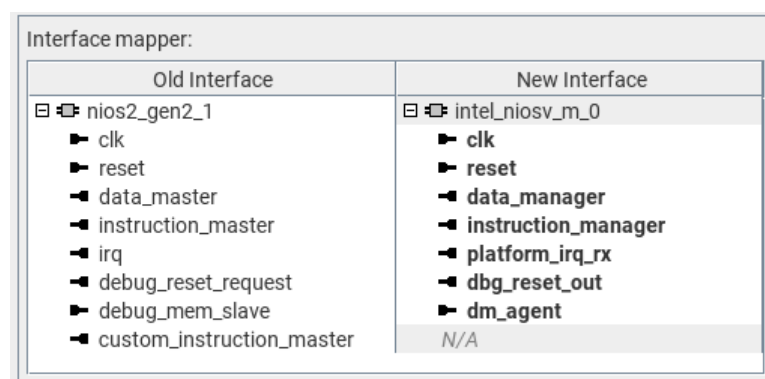
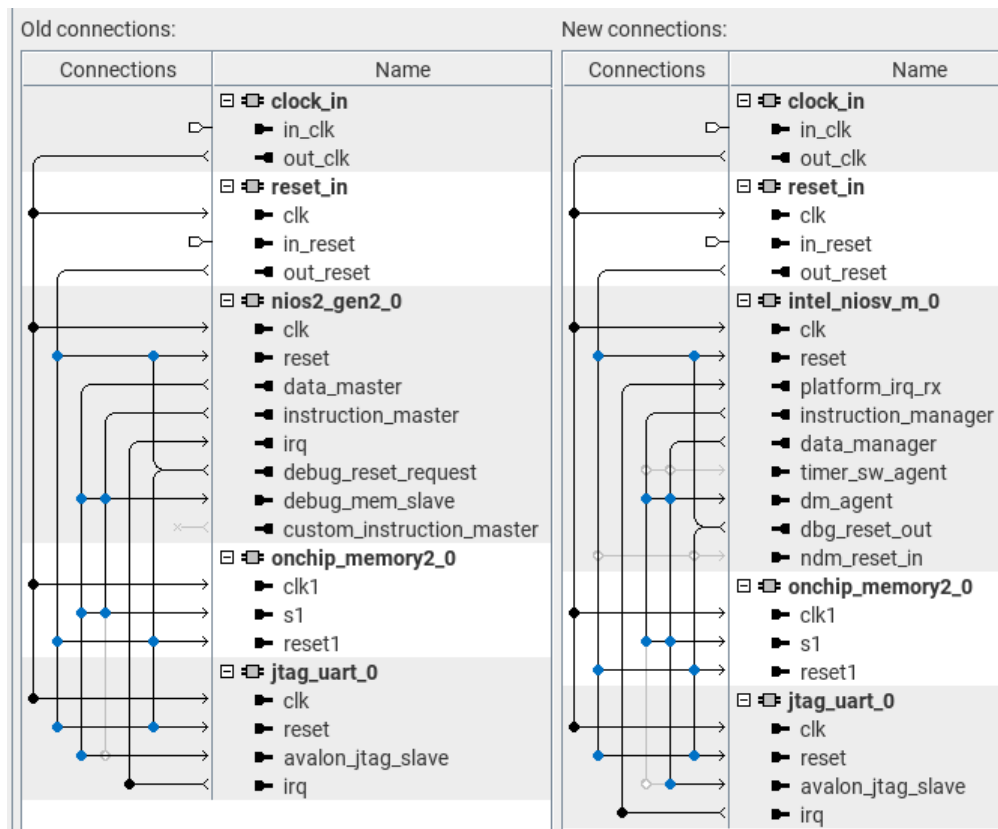


Figure 9. Old and New System Connections



7. Proceed to establish the following connections:
 - Disconnect dbg_reset_out to reset
 - Connect dbg_reset_out to ndm_reset_in
8. Next, verify the connections based on the table below:

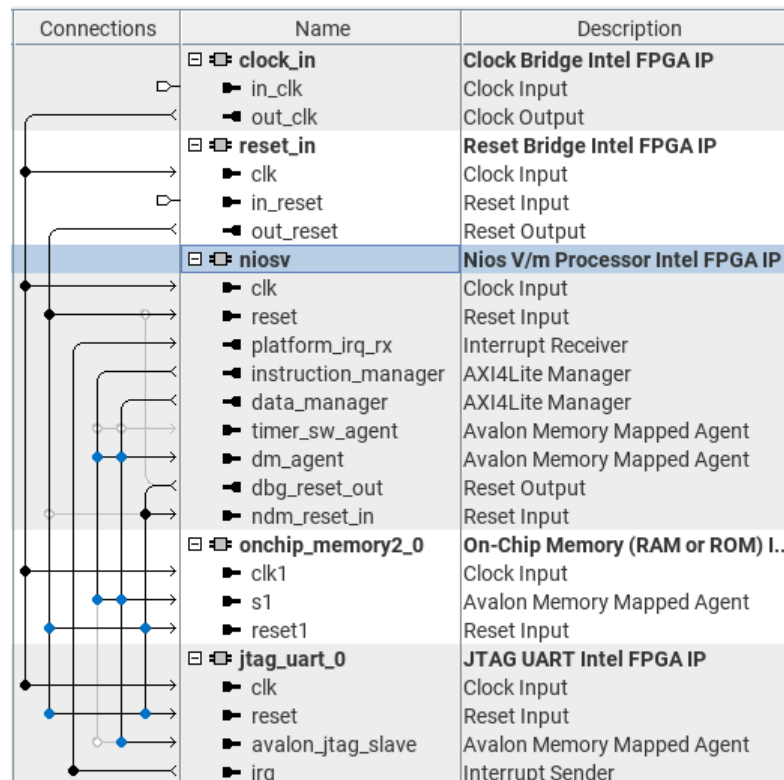
Table 8. Nios V Processor to Nios II Processor Interfaces

Interface	Nios V Processor	Nios II Processor	Description
Clock	clock	clock	Similar clock input
Reset	reset	reset	Similar hard reset
Debug reset	- dbg_reset_out - ndm_reset_in	debug_reset_request	Mainly used to issue debug reset from CLI or IDE. By default, this feature is enabled in Nios II/e processor but disabled in Nios V/m processor.
Interrupts	platform_irq_rx	irq	Nios II/e processor supports 32 platform interrupts. Nios V/m processor supports 16 platform interrupts.
Timer and Software Interrupt Module	timer_sw_agent	N/A	Only supported in Nios V processor. By default, it is connected to the data manager.
continued...			

Interface	Nios V Processor	Nios II Processor	Description
Debug	dm_agent	debug_mem_slave	Allow JTAG download and debug. For Nios V/m processor, it is connected to the data manager and instruction manager by default.
Data bus	data_manager	data_master	- Nios II/e processor uses Avalon® memory-mapped interface. - Nios V/m processor uses Arm® AMBA® AXI-4 Lite.
Instruction bus	instruction_manager	instruction_master	- Nios II/e processor uses Avalon memory-mapped interface. - Nios V/m processor uses Arm AMBA AXI-4 Lite.

- Open your selected Nios V processor IP, click **Vectors** ► **Reset Agent**, and update the parameters as onchip_memory2_0.s1.
- Rename the processor core to niosv. A preview of the final system is shown below.

Figure 10. Nios V Processor System



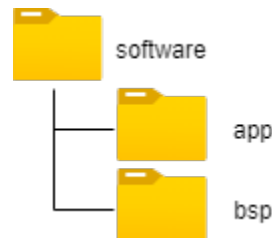
You can now generate the system HDL and compile Quartus Prime Project. Use the Quartus Prime Programmer software to configure the design SOF file for design verification.

Note: For Nios V/m and Nios V/c processors, you can enable **Enable Avalon Interface** to apply instruction and data bus as Avalon Memory Mapped Host. This feature is available starting Quartus Prime Pro Edition software version 24.3.

4.2.2.2. Software Component

The software development flow starts with Board Support Package (BSP) and application (APP) creation. Altera recommends you to create a software folder with app and bsp sub-folders inside your Quartus Prime project directory. Nios II processor software also practice similar file organization.

Figure 11. Nios Processor Software Project File Directory

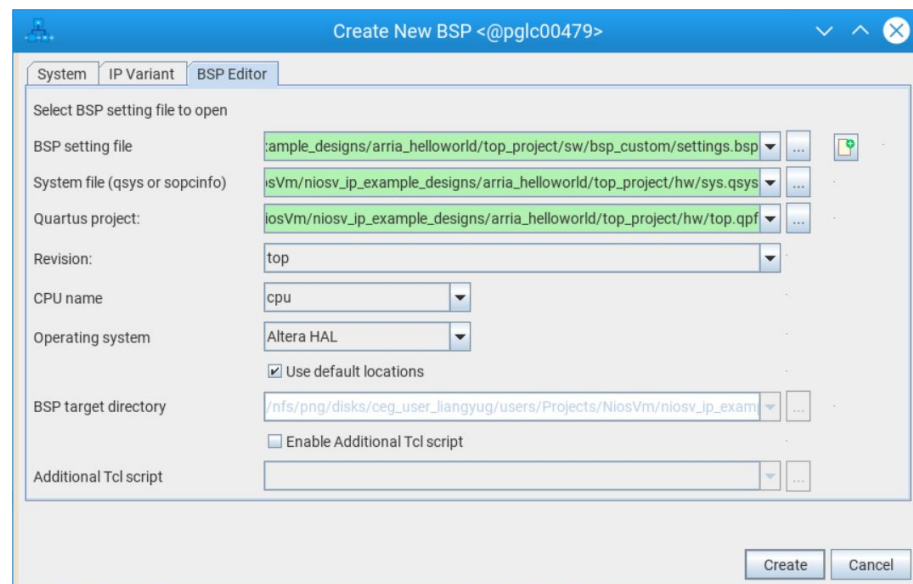


Note: Generated Nios II processor software files are not compatible with the Nios V processor. You need to regenerate the Nios V processor BSP and APP project.

Nios V Processor BSP Regeneration

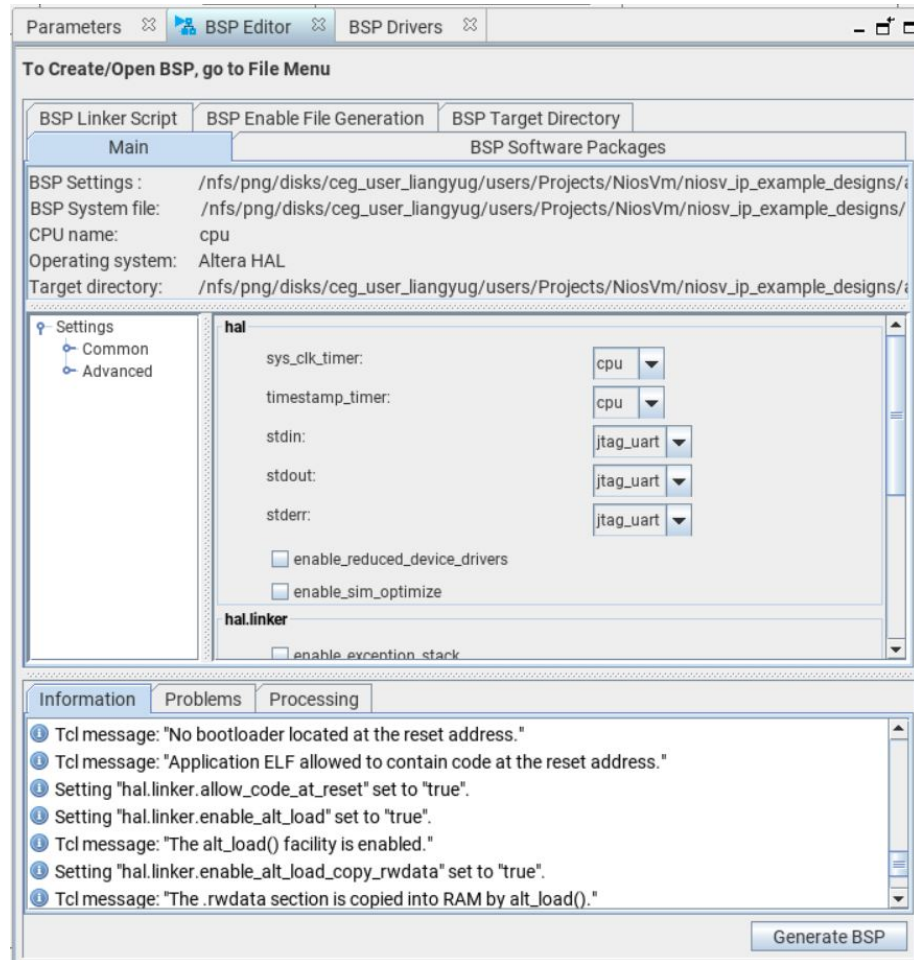
1. To open the BSP Editor, go to **Platform Design > File > New BSP**.
2. Set the **BSP Setting File** as settings.bsp. Select the destination folder to the BSP folder.
3. Select the design QSYS file in the **System file (qsys or sopcinfo)** field, then click **Create**.

Figure 12. Nios V Processor BSP Creation



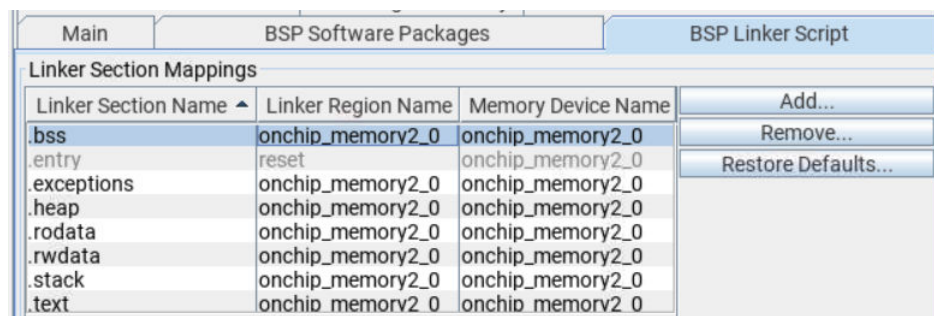
4. Apply the same Nios II processor BSP settings into the Nios V processor BSP settings. Refer to [Altera HAL Settings](#) to replicate the BSP Settings.

Figure 13. Nios V BSP Settings



5. Select the appropriate memory region for exception vector in **BSP Editor** ➤ **Linker Script**

Figure 14. Linker Script



6. Click **Generate BSP** to create a new BSP project for Nios V processor.

Nios V Processor APP Generation

1. Copy the Nios II application source codes into the APP folder.
2. Start the Nios V Command Shell.
3. Run `niosv-app` command to create an APP `CMakeLists.txt` file.

```
niosv-app -b=<BSP directory> -a=<APP directory> -s=<Nios II application  
source codes>
```

4. Run CMake executable.

```
cmake -B <APP directory>/build -S <APP directory> -G "Unix Makefiles"
```

5. Run Make executable. After successful Make, the Nios V processor software ELF file is located in the `<APP directory>/build` folder.

```
make -C <APP directory>/build
```

6. Download the software ELF file into the Nios V processor system.

```
niosv-download <Nios V ELF file>
```

7. Open JTAG UART terminal to print the Hello World message.

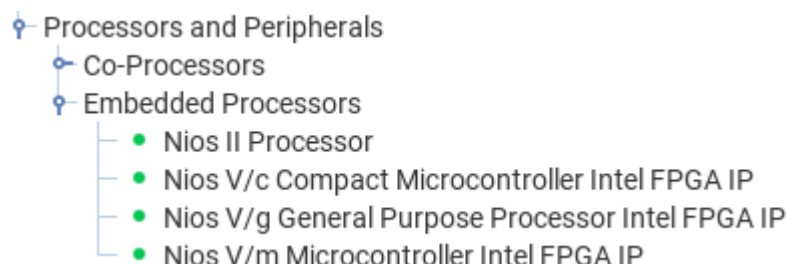
```
juart-terminal
```

Alternatively, you can use Ashling RiscFree IDE for Altera FPGAs for Nios V processor software development. After the BSP creation and APP creation, you can import both folder into the workspace of Ashling RiscFree IDE for Altera FPGAs to continue.

5. Processor Core Comparison

You can instantiate the Nios II and Nios V processors IP from **IP Catalog > Processors and Peripherals > Embedded Processors**.

Figure 15. IP Catalog



5.1. Upgrading Nios II/e Processor to Nios V/c Processor

5.1.1. Hardware Perspective

5.1.1.1. Signal and Interface Mapping

Table 9. Primary Interface

Interface	Nios V Processor	Nios II Processor	Description
Clock	clock	clock	Similar clock input
Reset	reset	reset	Similar hard reset
Debug reset	—	debug_reset_request	Mainly used to issue debug reset from CLI or IDE. By default, this feature is enabled in Nios II/e processor. However, Nios V/c is not supporting debug features.
Interrupts	—	irq	Nios II/e processor supports 32 platform interrupts. Nios V/c processor does not support interrupts.
Debug	—	debug_mem_slave	Allow JTAG download and debug. Nios V/c processor does not support debug features.
<i>continued...</i>			

Interface	Nios V Processor	Nios II Processor	Description
Data bus	data_manager	data_master	- Nios II/e processor uses Avalon memory-mapped interface. - Nios V/c processor uses Arm* AMBA* AXI-4 Lite by default. You can switch to Avalon memory-mapped interface.
Instruction bus	instruction_manager	instruction_master	- Nios II/e processor uses Avalon memory-mapped interface. - Nios V/c processor uses Arm AMBA AXI-4 Lite by default. You can switch to Avalon memory-mapped interface.
Custom instruction	—	custom_instruction_master	Nios V/c does not support custom instruction.

Table 10. Optional Interface

Interface	Nios V Processor	Nios II Processor	Description
Debug request	—	- debugreq - debugack	Nios V/c processor does not support debug requests.
CPU reset request	- cpu_resetreq_resetreq - cpu_resetreq_ack	- cpu_resetrequest - cpu_resettaken	Same application
ECC event	cpu_ecc_status	ecc_event_bus	Nios V/c processor supports ECC on embedded memory blocks of the core. Nios V processor supports error detection and status reporting, but not ECC recovery.

5.1.1.2. IP Parameter Configuration

In the Nios V/c processor, the following parameters are provided for you to configure:

- **CPU Architecture**—Apply the Avalon Memory Mapped instruction and data buses. When disabled, the buses are Arm* AMBA* AXI-4 Lite.
- **Use Reset Request**—Exposes the reset request interface. You can enable this optional interface based on your needs requirement.
- **Reset Agent and Reset Offset**—Configures the processor reset vector, equivalent to **Reset Vector and Reset Offset** in Nios II/e processor.
- **ECC**—Nios V/c processor supports ECC Error Detection and Status Reporting on embedded memory blocks within the core.

The figures below display the configuration settings of the Nios V/c processor.

Figure 16. Nios V/c Processor – IP Parameter Editor

Table 11. IP Parameter Editor Configuration

IP Parameter Editor	Nios V/c Processor	Nios II/e Processor
Reset Vector	Navigate to Vectors .	Navigate to, Vector > Reset Vector
Exception Vector	Define as .exceptions in BSP Editor > Linker Section Mappings	Navigate to, Vector > Exception Vector
JTAG Debug	—	Navigate to, JTAG Debug > JTAG Debug Settings
ECC	Navigate to ECC	Navigate to Advanced Features > General > ECC Present
CPU Reset Request	Navigate to Use Reset Request	Navigate to Advanced Features > General > Include cpu_resetrequest and cpu_resettaken signals
CPUID Value	—	Navigate to Advanced Features > General > CPUID control register value
Generate Trace File	—	Navigate to Advanced Features > General > Generate trace file during RTL simulation
RAM Memory Protection	—	Navigate to Advanced Features RAM Memory Protection

The figures below display the configuration settings of the Nios II/e processor.

Figure 17. Nios II/e Processor – IP Parameter Editor (Vector Tab)

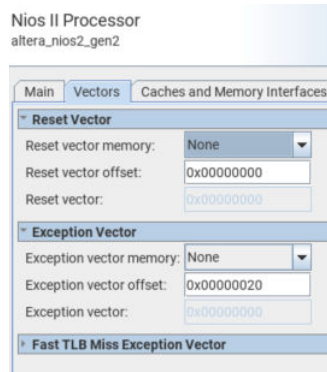


Figure 18. Nios II/e Processor – IP Parameter Editor (JTAG Debug Tab)

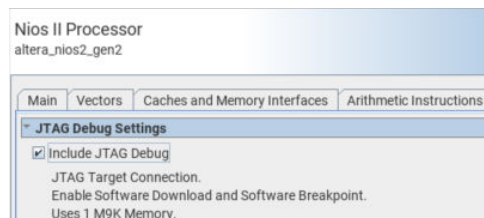
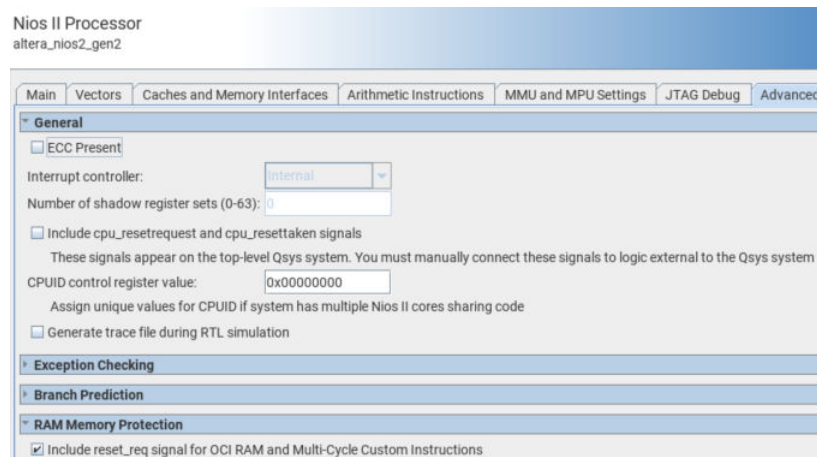


Figure 19. Nios II/e Processor – IP Parameter Editor (Advanced Features Tab)



5.1.1.3. BSP Project Generation

Generated BSP for Nios II and Nios V processors are not compatible. You are required to generate a new BSP project for Nios V processor using the BSP Editor, after applying the same Nios II processor BSP Settings. Refer to the settings in *Altera HAL BSP Settings* and *Micrium MicroC/OS-II BSP Settings*.

Nios V processor supports software-programmable exception address. Thus, exception vector configuration for Nios V processor is relocated into the BSP Editor. In Nios II processor, the exception vector configuration is found in the IP Parameter Editor.

The default exception region (agent) depends on the largest memory region connected to the Nios V processor. You can select the .exceptions region using the drop-down list, which is populated from all connected memory regions.

Figure 20. Linker Section Mapping

Linker Section Mappings			
Linker Section Name ▲	Linker Region Name	Memory Device Name	
.bss	bootcopier_ram	bootcopier_ram	Add...
.entry	reset	bootcopier_rom	Remove...
.exceptions	bootcopier ram ▼	bootcopier_ram	Restore Defaults...
.heap	reset	bootcopier_ram	
.rodata	bootcopier_rom	bootcopier_rom	
.rwddata	bootcopier_rom	bootcopier_rom	
.stack	bootcopier_ram	bootcopier_ram	
.text	user_application_mem	bootcopier rom	

For exception offset, you can configure the Offset (bytes) of the selected region. By default, the exception offset starts at 0. If the exception and reset agent are in the same memory region, the exception offset begins at 32 bytes after the reset offset.

Figure 21. Linker Memory Regions

Linker Memory Regions				
Linker Region Name	Address Range ▼	Memory Device Name	Size (bytes)	Offset (bytes)
user_application_mem	0x00F00000 - 0x00F3FFFF	user_application_mem	262144	0
bootcopier_ram	0x00090000 - 0x000917FF	bootcopier_ram	6144	0
bootcopier_rom	0x00040020 - 0x000417FF	bootcopier_rom	6112	32
reset	0x00040000 - 0x0004001F	bootcopier rom	32	0

Related Information

- [Altera HAL Settings](#) on page 61
- [Micrium MicroC/OS-II BSP Settings](#) on page 63

5.1.2. Software Perspective

5.1.2.1. Command-Line Build Flow

Similar with Nios II Software Build Tools, you can create the Nios V processor software project with Nios V Command Shell and Ashling RiscFree IDE for Altera FPGAs.

Table 12. CLI Build Flow Comparison

Flow	Nios V Processor	Nios II Processor
Invoke BSP Editor GUI	- For Quartus Prime Pro Edition software, use qsys-edit --1=.bsp. - For Quartus Prime Standard Edition software, use niosv-bsp-editor .	- For Quartus Prime Pro Edition software, use nios2-bsp-editor or qsys-edit --1=.bsp. <i>Note:</i> The BSP Editor is integrated into Platform Designer. - For Quartus Prime Standard Edition, use nios2-bsp-edit. <i>Note:</i> The BSP Editor is not integrated as part of Platform Designer.
Invoke Command Shell	niosv-shell	- In the Windows operating system, on the Start menu, point to Programs > Intel FPGA > Nios II EDS <version>, and click Nios II <version> Command Shell. - In the Linux operating system, in a command shell, change directories to <i><Nios II EDS install path></i>, and type the command nios2_command_shell.sh.
Create a BSP	niosv-bsp -c -p=<path to .qpf file> -s=<path to .qsys file> -t=<bsp type> [OPTIONS] settings.bsp	nios2-bsp <bsp type> <bsp-dir> [<sopc>] [<override>]
Generate a BSP	niosv-bsp -g [OPTIONS] settings.bsp	
Update a BSP	niosv-bsp -u [OPTIONS] settings.bsp	
Examine a BSP	niosv-bsp -q -E=<script name> [OPTIONS] settings.bsp	nios2-bsp-query-settings --settings settings.bsp
Create an APP	niosv-app -b=<bsp directory> -s=<source files directory> [OPTIONS]	nios2-app-generate-makefile --bsp-dir <bsp directory> --src-file <source files directory> [<OPTIONS>]
Build an App	cmake -B <build directory> -S <app directory> -G "Unix Makefiles" make -C <build directory>	make
Download an ELF	niosv-download <filename.elf>	nios2-download <filename.elf>
Run an ELF	niosv-download -g -r	nios2-download -g -r
Open JTAG UART Terminal	juart-terminal	nios2-terminal

Related Information

- [Nios II Software Developer Handbook](#)
- [Nios V Processor Software Developer Handbook](#)

5.1.2.2. IDE Build Flow

Nios V processor supports Ashling RiscFree IDE for Altera FPGAs while Nios II processor supports Nios II Embedded Design Suite. Both are Eclipse-based IDE with an identical graphic user interface (GUI).

Table 13. IDE Build Flow Comparison

Flow	Ashling RiscFree IDE for Altera FPGAs	Nios II EDS
Create a BSP	External tool configuration: niosv-bsp	Supported BSP Editor in GUI
Create an APP	External tool configuration: niosv-app	Supported GUI with design template
Build an ELF	Supported in GUI	Supported in GUI
Download an ELF	Supported in GUI	Supported in GUI
Open JTAG UART Terminal	External tool configuration: juart-terminal	Nios II Console

Related Information

Ashling* RiscFree* Integrated Development Environment (IDE) for Altera FPGAs User Guide

5.2. Upgrading Nios II Processor to Nios V/m Processor

5.2.1. Hardware Perspective

5.2.1.1. Signal and Interface Mapping

Table 14. Primary Interface

Interface	Nios V Processor	Nios II Processor	Description
Clock	clock	clock	Similar clock input
Reset	reset	reset	Similar hard reset
Debug reset	- dbg_reset_out - ndm_reset_in	debug_reset_request	Mainly used to issue debug reset from CLI or IDE. By default, this feature is enabled in Nios II/e processor but disabled in Nios V/m processor.
Interrupts	platform_irq_rx	irq	Nios II/e processor supports 32 platform interrupts. Nios V/m processor supports 16 platform interrupts.
Timer and Software Interrupt Module	timer_sw_agent	N/A	Only supported in Nios V processor. By default, it is connected to the data manager.
Debug	dm_agent	debug_mem_slave	Allow JTAG download and debug. For Nios V/m processor, it is connected to the data manager and instruction manager by default.
continued...			

Interface	Nios V Processor	Nios II Processor	Description
Data bus	data_manager	data_master	- Nios II/e processor uses Avalon memory-mapped interface - Nios V/m processor uses Arm AMBA AXI-4 by default. You can switch to Avalon memory-mapped interface.
Instruction bus	instruction_manager	instruction_master	- Nios II/e processor uses Avalon memory-mapped interface - Nios V/m processor uses Arm AMBA AXI-4 by default. You can switch to Avalon memory-mapped interface.
Custom instruction	—	custom_instruction_master	Nios V/m processor does not support custom instruction.

Table 15. Optional Interface

Interface	Nios V Processor	Nios II Processor	Description
Debug request	—	- debugreq - debugack	Nios V/m processor does not support debug requests.
CPU reset request	- cpu_resetreq_resetreq - cpu_resetreq_ack	- cpu_resetrequest - cpu_resettaken	Same application
ECC event	cpu_ecc_status	ecc_event_bus	Nios V/m processor supports ECC on embedded memory blocks of the core. Nios V processor supports error detection and status reporting, but not ECC recovery.

Related Information

[Nios V Processor Reference Manual](#)

For more information about the Nios V/m processor architecture.

5.2.1.2. IP Parameter Configuration

The following are the parameters that you need to configure in the Nios V/m processor:

- **Debug and Use Reset Request**—The **Nios V Processor IP Parameter Editor** exposes the debug and reset request interface. Depending on your requirement, you can enable these optional interfaces.
- **Reset Agent and Reset Offset**—Configures the processor reset vector. Equivalent to the **Reset Vector and Reset Offset** in the Nios II processors.
- **Interrupt Mode**—Supports Direct Interrupt and Vectored Interrupt.
- **CPU Architecture**—Instantiates the non-pipelined Nios V/m processor. It has smaller logic area, but lower IPC performance than pipelined. Optionally, you can apply the following configurations:
 - Apply the Avalon Memory Mapped instruction and data buses. When disabled, these buses use the Arm AMBA AXI-4 Lite protocol.
 - Assign a value to the Hart ID register (mhartid).
- **ECC**—Supports ECC error detection and status reporting on embedded memory blocks within the core.

The figure below display the configuration settings of the Nios V/m processor.

Figure 22. Nios V/m Microcontroller Intel® FPGA IP – IP Parameter Editor

Nios V/m Microcontroller Intel FPGA IP
intel_niosv_m

Example Designs
For available Nios V/m Example Designs, please go to: Quartus > File Menu > Open Example Project

Debug
☒ Enable Debug
☐ Enable Reset from Debug Module

Use Reset Request
☐ Add Reset Request Interface

Traps, Exceptions, and Interrupts
 Reset Agent:
 Reset Vector Offset:
 Interrupt Mode:

CPU Architecture
 When pipelining is enabled, you benefit from higher instructions per cycle (IPC), but at the cost of higher area and lower frequency. When pipelining is disabled, you benefit from lower area and higher frequency, but at the cost of lower IPC.
☒ Enable Pipelining in CPU
☐ Enable Avalon Interface
 mhartid CSR value:

ECC
☐ Enable Error Detection & ECC Status Reporting

Table 16. IP Parameter Editor Comparison

IP Parameter Editor	Nios V Processor	Nios II Processor
Reset Vector	Navigate to Traps, Exceptions, and Interrupts > Reset Vectors	Navigate to Vector > Reset Vector
Exception Vector	Define as .exceptions in BSP Editor > Linker Section Mappings .	Navigate to Vector > Exception Vector
Fast TLB Miss Exception Vector	—	Navigate to Vector > Fast TLB Miss Exception Vector
Cache and Memory Interfaces	—	Navigate to Caches and Memory Interfaces
Arithmetic Instructions	—	Navigate to Arithmetic Instructions
MMU and MPU Settings	—	Navigate to MMU and MPU Settings
JTAG Debug	Navigate to Debug > Enable Debug	Navigate to JTAG Debug > JTAG Debug Settings
Enable Reset from Debug Module	Navigate to Debug > Enable Reset from Debug Module	Enable when JTAG Debug is enabled
ECC	Navigate to ECC	Navigate to Advanced Features > General > ECC Present
Interrupt Controller	Navigate to Traps, Exceptions, and Interrupts > Interrupt Mode	Navigate to Advanced Features > Interrupt controller
Shadow Register	—	Navigate to Advanced Features > Number of shadow register sets
continued...		

IP Parameter Editor	Nios V Processor	Nios II Processor
CPU Reset Request	Navigate to Use Reset Request	Navigate to Advanced Features > General > Include cpu_resetrequest and cpu_resettaken signals
CPUID Value	Navigate to CPU Architecture > mhartid CSR value	Navigate to Advanced Features > General > CPUID control register value
Generate Trace File	—	Navigate to Advanced Features > General > Generate trace file during RTL simulation
Exception Checking	Enabled by default	Navigate to Advanced Features > Exception Checking
Branch Prediction	—	Navigate to Advanced Features > Branch Prediction
RAM Memory Protection	—	Navigate to Advanced Features > RAM Memory Protection
Non-pipelined Configuration	Navigate to CPU Architecture > Enable Pipelining in CPU	—

The following figures display the configuration settings of both Nios II/e and Nios II/f processor.

Figure 23. Nios II Processor – IP Parameter Editor (Vector Tab)

Nios II Processor
altera_nios2_gen2

Main Vectors Caches and Memory Interfaces Arithmetic

Reset Vector

Reset vector memory: None

Reset vector offset: 0x00000000

Reset vector: 0x00000000

Exception Vector

Exception vector memory: None

Exception vector offset: 0x00000020

Exception vector: 0x00000000

Fast TLB Miss Exception Vector

Fast TLB Miss Exception vector memory: None

Fast TLB Miss Exception vector offset: 0x00000000

Fast TLB Miss Exception vector: 0x00000000

Figure 24. Nios II Processor – IP Parameter Editor (Caches and Memory Interfaces Tab)

Nios II Processor
altera_nios2_gen2

Main Vectors Caches and Memory Interfaces Arithmetic Instructions

Instruction Cache

Size: 2 Kbytes

Add burstcount signal to instruction_master: Disable

Flash Accelerator

Line Size: None

Number of Cache Lines: 2

Data Cache

Size: 2 Kbytes

Victim buffer implementation: RAM

Add burstcount signal to data_master: Disable

☒ Use most-significant address bit in processor to bypass data cache

Tightly-coupled Memories

Number of tightly coupled instruction master ports: None

Number of tightly coupled data master ports: None

Peripheral Region

Size: None

Base Address: 0x00000000

Figure 25. Nios II Processor – IP Parameter Editor (Arithmetic Instructions Tab)

Nios II Processor
altera_nios2_gen2

Main Vectors Caches and Memory Interfaces Arithmetic Instructions

Multiply/Shift/Rotate Hardware: Auto Selection

Divide Hardware: None

Arithmetic Implementation

Multiply Implementation: 3 16-bit multipliers

Multiply Extended Implementation: None

Shift/Rotate Implementation: Logic elements (pipelined)

Figure 26. Nios II Processor – IP Parameter Editor (MMU and MPU Settings Tab)

Nios II Processor
altera_nios2_gen2

Main Vectors Caches and Memory Interfaces Arithmetic Instructions

MMU

☐ Include MMU

Process ID (PID) bits: 8 Bits

☒ Optimize TLB entries base on device family

TLB entries: 128 Entries

TLB Set-Associativity: 16 Ways

Micro DTLB entries: 6 Entries

Micro ITLB entries: 4 Entries

MPU

☐ Include MPU

☐ Use Limit for region range

Number of data regions: 8

Minimum data region size: 4 Kbytes

Number of instruction regions: 8

Minimum instruction region size: 4 Kbytes

Figure 27. Nios II Processor – IP Parameter Editor (JTAG Debug Tab)

Nios II Processor
altera_nios2_gen2

Main Vectors Caches and Memory Interfaces Arithmetic Instructions MMIO

JTAG Debug Settings

☒ Include JTAG Debug

JTAG Target Connection.
Enable Software Download and Software Breakpoint.
Uses 1 M9K Memory.

Hardware Breakpoints: 0

Data Triggers: 0

Trace Types: None

Trace Storage: On-Chip Trace

Onchip Trace Frame Size: 128

☐ Include debugreq and debugack Signals

These signals appear on the top-level Qsys system.
You must manually connect these signals to logic external to the Qsys system.

Figure 28. Nios II Processor – IP Parameter Editor (Advanced Features Tab)

Related Information

[Nios V Embedded Processor Design Handbook](#)

5.2.1.3. BSP Project Generation

Generated BSP for Nios II and Nios V processors are not compatible. You are required to generate a new BSP project for Nios V processor using the BSP Editor, after applying the same Nios II processor BSP Settings. Refer to the settings in *Altera HAL BSP Settings* and *Micrium MicroC/OS-II BSP Settings*.

Nios V processor supports software-programmable exception address. Thus, exception vector configuration for Nios V processor is relocated into the BSP Editor. In Nios II processor, the exception vector configuration is found in the IP Parameter Editor.

The default exception region (agent) depends on the largest memory region connected to the Nios V processor. You can select the .exceptions region using the drop-down list, which is populated from all connected memory regions.

Figure 29. Linker Section Mapping

Linker Section Mappings			
Linker Section Name ▲	Linker Region Name	Memory Device Name	Add...
.bss	bootcopier_ram	bootcopier_ram	Remove...
.entry	reset	bootcopier_rom	Restore Defaults...
.exceptions	bootcopier_ram	bootcopier_ram	
.heap	reset	bootcopier_ram	
.rodata	bootcopier_rom	bootcopier_rom	
.rwdata	bootcopier_rom	bootcopier_rom	
.stack	bootcopier_ram	bootcopier_ram	
.text	user_application_mem	bootcopier_rom	

For exception offset, you can configure the Offset (bytes) of the selected region. By default, the exception offset starts at 0. If the exception and reset agent are in the same memory region, the exception offset begins at 32 bytes after the reset offset.

Figure 30. Linker Memory Regions

Linker Memory Regions					
Linker Region Name	Address Range ▼	Memory Device Name	Size (bytes)	Offset (bytes)	Add...
user_application_mem	0x00F00000 - 0x00F3FFFF	user_application_mem	262144	0	Remove...
bootcopier_ram	0x00090000 - 0x000917FF	bootcopier_ram	6144	0	Restore Defaults...
bootcopier_rom	0x00040020 - 0x000417FF	bootcopier_rom	6112	32	
reset	0x00040000 - 0x0004001F	bootcopier_rom	32	0	Add Memory Devi...
					Remove Memory ...
					Memory Usage...
					Memory Map...

Related Information

- [Altera HAL Settings](#) on page 61
- [Micrium MicroC/OS-II BSP Settings](#) on page 63

5.2.2. Software Perspective

Nios II and Nios V processors have different software build tools but serve the same functionality to build software projects.

Table 17. Nios V Processor Software Differences

Tools	Nios V Processor	Nios II Processor
Command Shell	Nios V Command Shell	Nios II Command Shell
Command Line	Nios V Processor Command Line Utilities	Nios II Software Build Tools Utilities
Development IDE	Ashling RiscFree IDE for Altera FPGAs	Nios II EDS with Eclipse IDE

5.2.2.1. Command-Line Build Flow

Similar with Nios II Software Build Tools, you can create the Nios V processor software project with Nios V Command Shell and Ashling RiscFree IDE for Altera FPGAs.

Table 18. CLI Build Flow Comparison

Flow	Nios V Processor	Nios II Processor
Invoke BSP Editor GUI	- For Quartus Prime Pro Edition software, use qsys-edit --1=.bsp. - For Quartus Prime Standard Edition software, use niosv-bsp-editor .	- For Quartus Prime Pro Edition software, use nios2-bsp-editor or qsys-edit --1=.bsp. <i>Note:</i> The BSP Editor is integrated into Platform Designer. - For Quartus Prime Standard Edition, use nios2-bsp-edit. <i>Note:</i> The BSP Editor is not integrated as part of Platform Designer.
Invoke Command Shell	niosv-shell	- In the Windows operating system, on the Start menu, point to Programs > Intel FPGA > Nios II EDS <version>, and click Nios II <version> Command Shell:. - In the Linux operating system, in a command shell, change directories to <i><Nios II EDS install path></i>, and type the command nios2_command_shell.sh.
Create a BSP	niosv-bsp -c -p=<path to .qpf file> -s=<path to .qsys file> -t=<bsp type> [OPTIONS] settings.bsp	nios2-bsp <bsp type> <bsp-dir> [<sopc>] [<override>]
Generate a BSP	niosv-bsp -g [OPTIONS] settings.bsp	
Update a BSP	niosv-bsp -u [OPTIONS] settings.bsp	
Examine a BSP	niosv-bsp -q -E=<script name> [OPTIONS] settings.bsp	nios2-bsp-query-settings --settings settings.bsp
Create an APP	niosv-app -b=<bsp directory> -s=<source files directory> [OPTIONS]	nios2-app-generate-makefile --bsp-dir <bsp directory> --src-file <source files directory> [<OPTIONS>]
Build an App	cmake -B <build directory> -S <app directory> -G "Unix Makefiles" make -C <build directory>	make
Download an ELF	niosv-download <filename.elf>	nios2-download <filename.elf>
Run an ELF	niosv-download -g -r	nios2-download -g -r
Open JTAG UART Terminal	juart-terminal	nios2-terminal

Related Information

- [Nios II Software Developer Handbook](#)
- [Nios V Processor Software Developer Handbook](#)

5.2.2.2. IDE Build Flow

Nios V processor supports Ashling RiscFree IDE for Altera FPGAs while Nios II processor supports Nios II Embedded Design Suite. Both are Eclipse-based IDE with an identical graphic user interface (GUI).

Table 19. IDE Build Flow Comparison

Flow	Ashling RiscFree IDE for Altera FPGAs	Nios II EDS
Create a BSP	External tool configuration: niosv-bsp	Supported BSP Editor in GUI
Create an APP	External tool configuration: niosv-app	Supported GUI with design template
Build an ELF	Supported in GUI	Supported in GUI
Download an ELF	Supported in GUI	Supported in GUI
Open JTAG UART Terminal	External tool configuration: juart-terminal	Nios II Console

Related Information

Ashling* RiscFree* Integrated Development Environment (IDE) for Altera FPGAs User Guide

5.3. Upgrading Nios II/f Processor to Nios V/g Processor

5.3.1. Hardware Perspective

5.3.1.1. Signal and Interface Mapping

Table 20. Primary Interface

Interface	Nios V Processor	Nios II Processor	Description
Clock	clock	clock	Similar clock input
Reset	reset	reset	Similar hard reset
Debug reset	- dbg_reset_out - ndm_reset_in	debug_reset_request	Mainly used to issue debug reset from CLI or IDE. By default, this feature is enabled in Nios II/f processor but disabled in Nios V/g processor.
Interrupts	platform_irq_rx	irq	Nios II/f processor supports 32 platform interrupts. Nios V/g processor supports 16 platform interrupts.
Timer and Software Interrupt Module	timer_sw_agent	—	Only supported in Nios V processor. By default, it is connected to the data manager.
Debug	dm_agent	debug_mem_slave	Allow JTAG download and debug. For Nios V/g processor, by default, it is connected to the data manager and instruction manager.

continued...

Interface	Nios V Processor	Nios II Processor	Description
Data bus	data_manager	data_master	- Nios II/f uses Avalon memory-mapped interface - Nios V/g processor uses Arm AMBA AXI-4 by default.
Instruction bus	instruction_manager	instruction_manager	- Nios II/f uses Avalon memory-mapped interface - Nios V/g processor uses Arm AMBA AXI-4 by default.
Custom instruction	— (Instantiate in the Nios V Processor IP Parameter Editor)	custom_instruction_master	- Nios II/f implements custom instruction as primary interface. - Nios V/g implements custom instruction as optional interface.

Table 21. Optional Interface

Interface	Nios V Processor	Nios II Processor	Description
Debug request	—	- debugreq - debugack	Nios V/g processor does not support debug requests.
CPU reset request	- cpu_resetreq_resetreq - cpu_resetreq_ack	- cpu_resetrequest - cpu_resettaken	Same application
ECC event	cpu_ecc_status	ecc_event_bus	Same application
Custom instruction	ci_custom<opcode>_<funct7[6:4]>	—	- Nios II/f implements custom instruction as primary interface. - Nios V/g implements custom instruction as optional interface.
Tightly coupled memory (TCM)	- instruction_tcs - data_tcs	tightly_coupled_master	Refer to the section <i>Tightly Coupled Memory</i> for more information.

Related Information

- [Nios V Processor Reference Manual](#)
For more information about the Nios V/m processor architecture.
- [Tightly Coupled Memory](#) on page 58

5.3.1.2. IP Parameter Configuration

You can configure the following parameters in the Nios V/g processor:

- CPU Architecture**—Nios V/g supports FPU as a built-in feature. No functional difference compared to Custom Instruction FPH1 or FPH2. Optionally, you can apply the following configurations:
 - Apply branch prediction for better performance.
 - Assign the Hart ID register value (mhartid).
- Debug** and **Use Reset Request**—Exposes the debug and reset request interface. Depending on your requirement, you can enable these optional interfaces.
- Lockstep** — Enable Nios V processor Lockstep feature for functional safety-critical applications.

- **Reset Agent and Reset Offset**—Configures the processor reset vector, equivalent to **Reset Vector and Reset Offset** in Nios II/f processor.
- **Enable Core Level Interrupt Controller**—Enable or disable the CLIC to support pre-emption and configurable interrupt trigger condition.
- **Interrupt Mode**—Supports Direct Interrupt and Vectored Interrupt and CLIC Interrupt (when Enable Core Level Interrupt Controller is enabled).
- **Shadow Register Files**—Enable shadow register to reduce the processor's context switching duration upon interrupt.
- **Caches**—Enable or disable the caches based on your design preference.
 - If you enable the cache, a minimum of 1 Kbyte of cache is instantiated for both data and instruction lines.
 - To disable the cache, set it to **No Cache**.
- **Peripheral Regions**—Assign these regions to achieve non-cacheable transactions with peripherals, to bypass the caches.
Note: You must place all peripherals within the peripheral region for correct system operation.
- **Tightly Coupled Memories**—Nios V/g processor supports up to 4 built-in TCMs, accessible by other host via AXI4-Lite. Nios II/f has 8 TCMs (4 instruction and 4 data) ports that need to be connected to on-chip memory.
- **ECC**—Nios V/g processor supports ECC error detection, status reporting, and single bit correction on embedded memory blocks within the core.
- **Custom Instructions**—Use the **Hardware Interface Table** and **Software CMacro Table** to instantiate the custom instruction.

Figure 31. Nios V/g General Purpose Processor – IP Parameter Editor (Part 1)

Nios V/g General Purpose Processor Intel FPGA IP
intel_niosv_g

Example Designs

CPU Architecture

☐ Enable Floating Point Unit

☒ Enable Branch Prediction

☐ Disable FSQRT & FDIV instructions for FPU

mhartid CSR value:

Debug

☒ Enable Debug

☐ Enable Reset from Debug Module

Lockstep

☐ Enable Lockstep

Blind Window Period:

Default Timeout Period:

☐ Enable Extended Reset Interface

Use Reset Request

☐ Add Reset Request Interface

Figure 32. Nios V/g General Purpose Processor – IP Parameter Editor (Part 2) with Enable Core Level Interrupt Controller Turned Off

Traps, Exceptions, and Interrupts

Reset Agent:

Reset Vector Offset:

☐ Enable Core Level Interrupt Controller

Interrupt Mode:

Shadow Register Files:

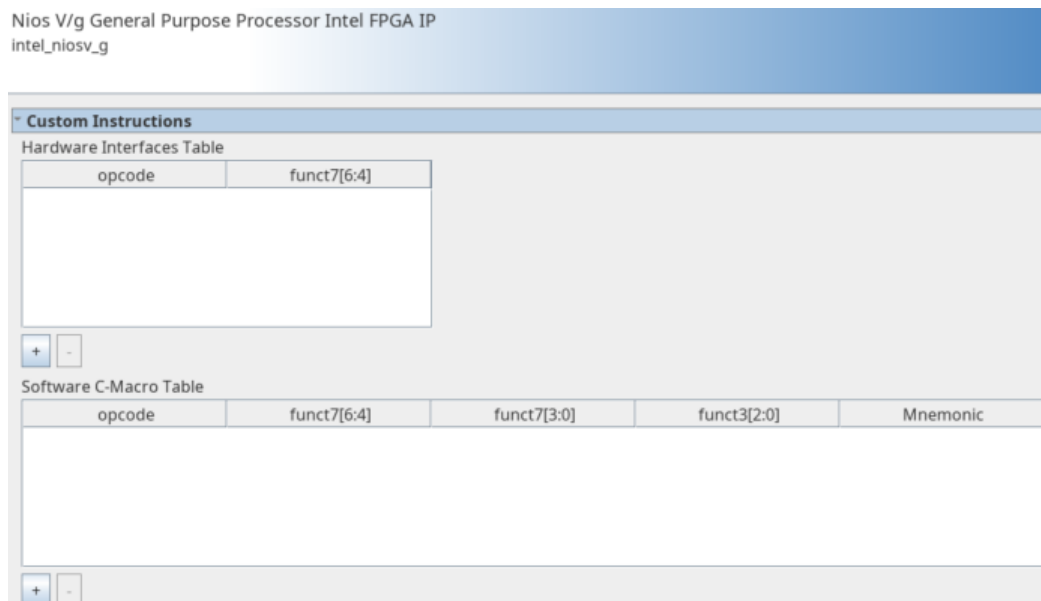
Figure 33. Nios V/g General Purpose Processor – IP Parameter Editor (Part 2) with Enable Core Level Interrupt Controller Turned On

Traps, Exceptions, and Interrupts	
Reset Agent:	intel_onchip_memory_0.s1
Reset Vector Offset:	0x00000000
☒ Enable Core Level Interrupt Controller	
Interrupt Mode:	Direct
Shadow Register Files:	Number of CLIC interrupt levels
Number of Platform interrupt sources:	16
CLIC Vector Table Alignment:	128 bytes
Number of Interrupt Levels:	2
Number of Interrupt Priorities per level:	8
☐ Configurable interrupt polarity	
☐ Support edge triggered interrupts	

Figure 34. Nios V/g General Purpose Processor – IP Parameter Editor (Part 3)

Nios V/g General Purpose Processor Intel FPGA IP
intel_niosv_g

Memory Configurations	
Caches	
Data Cache Size:	4 KBytes
Instruction Cache Size:	4 KBytes
Peripheral Regions	
Peripheral Region A Peripheral Region B	
Peripheral Region A Size:	0
Peripheral Region A Base Address:	0x00000000
Tightly Coupled Memories	
Instruction TCM1 Instruction TCM2 Data TCM1 Data TCM2	
Instruction TCM1 Size:	0
Instruction TCM1 Base Address:	0x00000000
Instruction TCM1 Initialization File:	
ECC	
☐ Enable Error Detection & ECC Status Reporting	
☐ Enable Single Bit Correction	

Figure 35. Nios V/g General Purpose Processor – IP Parameter Editor (Part 4)**Table 22. IP Parameter Editor Comparison**

IP Parameter Editor	Nios V/g General Purpose Processor	Nios II/f Processor
Reset Vector	Navigate to Traps, Exceptions, and Interrupts > Reset Vectors	Navigate to Vector > Reset Vector
Exception Vector	Define as .exceptions in BSP Editor > Linker Section Mappings .	Navigate to Vector > Exception Vector
Fast TLB Miss Exception Vector	—	Navigate to Vector > Fast TLB Miss Exception Vector
Instruction Cache	Navigate to Caches > Instruction Cache Size	Navigate to Caches and Memory Interfaces > Instruction Cache
Flash Accelerator	—	Navigate to Caches and Memory Interfaces > Flash Accelerator
Data Cache	Navigate to Caches > Data Cache Size	Navigate to Caches and Memory Interfaces > Data Cache
Tightly-Coupled Memory	Navigate to Tightly Coupled Memories	Navigate to Caches and Memory Interfaces > Tightly-coupled Memory
Peripheral Region	Navigate to Peripheral Regions	Navigate to Caches and Memory Interfaces > Peripheral Region
Arithmetic Instructions	Built-in multiply and divide instructions	Navigate to Arithmetic Instructions
MMU and MPU Settings	—	Navigate to MMU and MPU Settings
JTAG Debug	Navigate to Debug > Enable Debug	Navigate to JTAG Debug > JTAG Debug Settings
Enable Reset from Debug Module	Navigate to Debug > Enable Reset from Debug Module	Enable when JTAG Debug is enabled.
ECC	Navigate to ECC	Navigate to Advanced Features > General > ECC Present
continued...		

IP Parameter Editor	Nios V/g General Purpose Processor	Nios II/f Processor
Non-vectorized Interrupt Controller	Navigate to Traps, Exceptions, and Interrupts > Interrupt Mode > Direct	Navigate to Advanced Features > Interrupt controller > Internal
Vectorized Interrupt Controller	Navigate to Traps, Exceptions, and Interrupts > Interrupt Mode > Vectorized	Navigate to Advanced Features > Interrupt controller > EIC
Interrupt Controller with Pre-emption	Navigate to Traps, Exceptions, and Interrupts • Enable Core Level Interrupt Controller • Interrupt Mode > CLIC	Navigate to Advanced Features > Interrupt controller > EIC
Interrupt Controller with Configurable Trigger Condition	Navigate to Traps, Exceptions, and Interrupts • Enable Core Level Interrupt Controller • Interrupt Mode > CLIC	—
Shadow Register	Navigate to Traps, Exceptions, and Interrupts > Shadow Register Files	Navigate to Advanced Features > Number of shadow register sets
CPU Reset Request	Navigate to Use Reset Request	Navigate to Advanced Features > General > Include cpu_resetrequest and cpu_resettaken signals
CPUID Value	Navigate to CPU Architecture > mhartid CSR value	Navigate to Advanced Features > General > CPUID control register value
Generate Trace File	—	Navigate to Advanced Features > General > Generate trace file during RTL simulation
Exception Checking	Enabled by default	Navigate to Advanced Features > Exception Checking
Branch Prediction	Navigate to CPU Architecture > Enable Branch Prediction	Navigate to Advanced Features > Branch Prediction
RAM Memory Protection	—	Navigate to Advanced Features > RAM Memory Protection
Floating-Point Unit	Navigate to CPU Architecture > Enable Floating Point Unit	—
Custom Instructions	Navigate to Custom Instruction	—
Disable FSQRT and FDIV instruction for FPU	Navigate to CPU Architecture > > Disable FSQRT & FDIV instructions for FPU	—
Lockstep	Navigate to Lockstep	—

The following figures display the configuration settings of Nios II/f processor.

Figure 36. Nios II/f Processor – IP Parameter Editor (Vector Tab)

Nios II Processor
altera_nios2_gen2

Main Vectors Caches and Memory Interfaces Arithmetic

▼ **Reset Vector**

Reset vector memory: None

Reset vector offset: 0x00000000

Reset vector: 0x00000000

▼ **Exception Vector**

Exception vector memory: None

Exception vector offset: 0x00000020

Exception vector: 0x00000000

▼ **Fast TLB Miss Exception Vector**

Fast TLB Miss Exception vector memory: None

Fast TLB Miss Exception vector offset: 0x00000000

Fast TLB Miss Exception vector: 0x00000000

Figure 37. Nios II/f Processor – IP Parameter Editor (Caches and Memory Interfaces Tab)

Nios II Processor
altera_nios2_gen2

Main Vectors Caches and Memory Interfaces Arithmetic Instruction

Instruction Cache

Size: 2 Kbytes

Add burstcount signal to instruction_master: Disable

Flash Accelerator

Line Size: None

Number of Cache Lines: 2

Data Cache

Size: 2 Kbytes

Victim buffer implementation: RAM

Add burstcount signal to data_master: Disable

☒ Use most-significant address bit in processor to bypass data cache

Tightly-coupled Memories

Number of tightly coupled instruction master ports: None

Number of tightly coupled data master ports: None

Peripheral Region

Size: None

Base Address: 0x00000000

Figure 38. Nios II/f Processor – IP Parameter Editor (Arithmetic Instructions Tab)

Nios II Processor
altera_nios2_gen2

Main Vectors Caches and Memory Interfaces Arithmetic Instructions

Multiply/Shift/Rotate Hardware: Auto Selection

Divide Hardware: None

Arithmetic Implementation

Multiply Implementation: 3 16-bit multipliers

Multiply Extended Implementation: None

Shift/Rotate Implementation: Logic elements (pipelined)

Figure 39. Nios II/f Processor – IP Parameter Editor (MMU and MPU Settings Tab)

Nios II Processor
altera_nios2_gen2

Main Vectors Caches and Memory Interfaces Arithmetic Instructions

MMU

☐ Include MMU

Process ID (PID) bits: 8 Bits

☒ Optimize TLB entries base on device family

TLB entries: 128 Entries

TLB Set-Associativity: 16 Ways

Micro DTLB entries: 6 Entries

Micro ITLB entries: 4 Entries

MPU

☐ Include MPU

☐ Use Limit for region range

Number of data regions: 8

Minimum data region size: 4 Kbytes

Number of instruction regions: 8

Minimum instruction region size: 4 Kbytes

Figure 40. Nios II/f Processor – IP Parameter Editor (JTAG Debug Tab)

Nios II Processor
altera_nios2_gen2

Main Vectors Caches and Memory Interfaces Arithmetic Instructions MMIO

JTAG Debug Settings

☒ Include JTAG Debug

JTAG Target Connection.
Enable Software Download and Software Breakpoint.
Uses 1 M9K Memory.

Hardware Breakpoints: 0

Data Triggers: 0

Trace Types: None

Trace Storage: On-Chip Trace

Onchip Trace Frame Size: 128

☐ Include debugreq and debugack Signals

These signals appear on the top-level Qsys system.
You must manually connect these signals to logic external to the Qsys system.

Figure 41. Nios II/f Processor – IP Parameter Editor (Advanced Features Tab)

Nios II Processor
altera_nios2_gen2

Main Vectors Caches and Memory Interfaces Arithmetic

General

☐ ECC Present

Interrupt controller: Internal

Number of shadow register sets (0-63): 0

☐ Include cpu_resetrequest and cpu_resettaken signals
These signals appear on the top-level Qsys system. You must

CPUID control register value: 0x00000000
Assign unique values for CPUID if system has multiple Nios II

☐ Generate trace file during RTL simulation

Exception Checking

☐ Misaligned memory access

Branch Prediction

Branch prediction type: Dynamic

Number of entries (2-bits wide): 256 Entries

RAM Memory Protection

☒ Include reset_req signal for OCI RAM and Multi-Cycle Custom

5.3.1.3. BSP Project Generation

Generated BSP for Nios II and Nios V processors are not compatible. You are required to generate a new BSP project for Nios V processor using the BSP Editor, after applying the same Nios II processor BSP Settings. Refer to the settings in *Altera HAL BSP Settings* and *Micrium MicroC/OS-II BSP Settings*.

Nios V processor supports software-programmable exception address. Thus, exception vector configuration for Nios V processor is relocated into the BSP Editor. In Nios II processor, the exception vector configuration is found in the IP Parameter Editor.

The default exception region (agent) depends on the largest memory region connected to the Nios V processor. You can select the .exceptions region using the drop-down list, which is populated from all connected memory regions.

Figure 42. Linker Section Mapping

Linker Section Mappings			
Linker Section Name ^	Linker Region Name	Memory Device Name	Add...
.bss	bootcopier_ram	bootcopier_ram	Remove...
.entry	reset	bootcopier_rom	Restore Defaults...
.exceptions	bootcopier_ram	bootcopier_ram	
.heap	reset	bootcopier_ram	
.rodata	bootcopier_rom	bootcopier_ram	
.rwdata	bootcopier_rom	bootcopier_ram	
.stack	bootcopier_ram	bootcopier_ram	
.text	user_application_mem	bootcopier_rom	

For exception offset, you can configure the Offset (bytes) of the selected region. By default, the exception offset starts at 0. If the exception and reset agent are in the same memory region, the exception offset begins at 32 bytes after the reset offset.

Figure 43. Linker Memory Regions

Linker Memory Regions					
Linker Region Name	Address Range ^	Memory Device Name	Size (bytes)	Offset (bytes)	Add...
user_application_mem	0x00F00000 - 0x00F3FFFF	user_application_mem	262144	0	Remove...
bootcopier_ram	0x00090000 - 0x000917FF	bootcopier_ram	6144	0	Restore Defaults...
bootcopier_rom	0x00040020 - 0x000417FF	bootcopier_rom	6112	32	
reset	0x00040000 - 0x0004001F	bootcopier_rom	32	0	Add Memory Devi...
					Remove Memory ...
					Memory Usage...
					Memory Map...

Related Information

- [Altera HAL Settings](#) on page 61
- [Micrium MicroC/OS-II BSP Settings](#) on page 63

5.3.2. Software Perspective

Nios II and Nios V processors have different software build tools but serve the same functionality to build software projects.

Table 23. Nios V Processor Software Differences

Tools	Nios V Processor	Nios II Processor
Command Shell	Nios V Command Shell	Nios II Command Shell
Command Line	Nios V Processor Command Line Utilities	Nios II Software Build Tools Utilities
Development IDE	Ashling RiscFree IDE for Altera FPGAs	Nios II EDS with Eclipse IDE

5.3.2.1. Command-Line Build Flow

Similar with Nios II Software Build Tools, you can create the Nios V processor software project with Nios V Command Shell and Ashling RiscFree IDE for Altera FPGAs.

Table 24. CLI Build Flow Comparison

Flow	Nios V Processor	Nios II Processor
Invoke BSP Editor GUI	niosv-bsp-editor (for Quartus Prime Standard Edition only)	nios2-bsp-editor
Invoke Command Shell	niosv-shell	- In the Windows operating system, on the Start menu, point to Programs > Intel FPGA > Nios II EDS <version>, and click Nios II <version> Command Shell. - In the Linux operating system, in a command shell, change directories to <i><Nios II EDS install path></i>, and type the command <code>nios2_command_shell.sh</code>.
Create a BSP	niosv-bsp -c -p=<path to .qpf file> -s=<path to .qsys file> -t=<bsp type> [OPTIONS] settings.bsp	nios2-bsp <bsp type> <bsp-dir> [<sopc>] [<override>]
Generate a BSP	niosv-bsp -g [OPTIONS] settings.bsp	
Update a BSP	niosv-bsp -u [OPTIONS] settings.bsp	
Examine a BSP	niosv-bsp -q -E=<script name> [OPTIONS] settings.bsp	nios2-bsp-query-settings --settings settings.bsp
Create an APP	niosv-app -b=<bsp directory> -s=<source files directory> [OPTIONS]	nios2-app-generate-makefile --bsp-dir <bsp directory> --src-file <source files directory> [<OPTIONS>]
Build an App	cmake -B <build directory> -S <app directory> -G "Unix Makefiles" make -C <build directory>	make
Download an ELF	niosv-download <filename.elf>	nios2-download <filename.elf>
Run an ELF	niosv-download -g -r	nios2-download -g -r
Open JTAG UART Terminal	juart-terminal	nios2-terminal

Related Information

- [Nios II Software Developer Handbook](#)
- [Nios V Processor Software Developer Handbook](#)

5.3.2.2. IDE Build Flow

Nios V processor supports Ashling RiscFree IDE for Altera FPGAs while Nios II processor supports Nios II Embedded Design Suite. Both are Eclipse-based IDEs with an identical graphic user interface (GUI).

Table 25. IDE Build Flow Comparison

Flow	Ashling RiscFree IDE for Altera FPGAs	Nios II EDS
Create a BSP	External tool configuration: niosv-bsp	Supported BSP Editor in GUI
Create an APP	External tool configuration: niosv-app	Supported GUI with design template
Build an ELF	Supported in GUI	Supported in GUI
Download an ELF	Supported in GUI	Supported in GUI
Open JTAG UART Terminal	External tool configuration: juart-terminal	Nios II Console

Related Information

[Ashling* RiscFree* Integrated Development Environment \(IDE\) for Altera FPGAs User Guide](#)

6. Feature Comparison

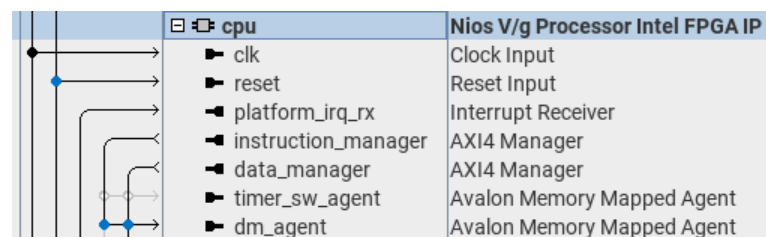
6.1. Timer

Nios V processor provides an internal timer functionality which allows you to configure your system clock timer and timestamp timer to use the internal timer. The Nios V processor internal timer is a 64-bit counter that increments at the processor's clock frequency. The timer removes the need for an additional timer component. The Nios II and Nios V processors are compatible with the legacy method using the `altera_avalon_timer` peripheral and HAL driver.

If the Nios II processor system uses system clock timer, timestamp timer or both, there are two migration options:

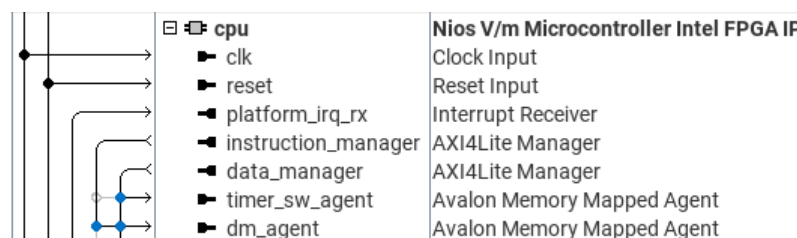
- Reuse the existing external Interval Timer. You can disable the Nios V processor Internal Timer as shown in the following figure (by disconnecting `timer_sw_agent` from `data_manager`).

Figure 44. Disabling Internal Timer



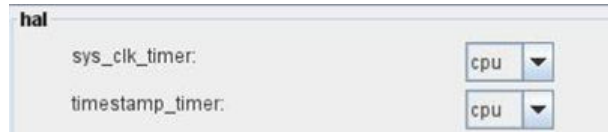
- Replace the existing external Interval Timer with Nios V processor Internal Timer. You can enable the Nios V processor Internal Timer as shown in the following figure (by connecting `timer_sw_agent` to `data_manager`), and remove the existing external Interval Timer.

Figure 45. Enabling Internal Timer



After that, you must select the internal timer as the new timer in the BSP settings.

Figure 46. Timer BSP Settings



Related Information

- [Nios V Processor Reference Manual](#)
For more information about timer and software interrupt module.
- [Nios V Processor Software Developer Handbook](#)
For more information about using timer devices.

6.2. Interrupt

Nios II processor offers Internal (IIC) or External Interrupt Controller (EIC) for interrupt handling, while Nios V processor provides the Core-Local Interruptor (CLINT) and Core-Local Interrupt Controller (CLIC). The difference results from the Instruction Set Architectures (ISAs) of the processors: Nios II and RISC-V.

Table 26. Comparison between Each Interrupt Controller

Implementation	Nios V Processor			Nios II Processor	
	CLINT-Direct	CLINT-Vectored	CLIC	IIC	EIC with Vectored Interrupt Controller
Interrupt type	Non-vectored	Vectored	Non-vectored	Non-vectored	Vectored
Hardware (Platform) interrupt	16 interrupts		16 to 2048 interrupts	32 interrupts	Unlimited
Software interrupt	1 interrupt			Not supported	Tied to each hardware interrupt.
Timer interrupt	1 interrupt			Implement as hardware interrupt with Interval Timer Core.	
Pre-emption with interrupt level	Not supported		Supported	Not supported	Supported
Priority arbitration	Not supported		Supported	Not supported	Supported
Register Set Switching with Shadow Register	Supported			Not supported	Supported
Configurable non-maskable interrupt	Implement using the global Machine Interrupt Enable bit in Machine Status Register (mstatus.mie)			Not supported	Supported
Configurable trigger condition	Positive level trigger only		• Positive level trigger • Rising edge trigger • Negative level trigger • Falling edge trigger	Positive level trigger only	
Legacy HAL Interrupt API	Not supported			Supported	
Enhanced HAL Interrupt API	Supported			Supported	

- The HAL driver for hardware interrupt is compatible, but take note that the Nios V processor accepts interrupts number up to 16 only which reflects the difference in interrupt bus width.
- Altera provides the HAL API for software interrupt and timer interrupt for the Nios V processor.

If you have custom device drivers, Altera recommends that you upgrade them to use the enhanced HAL interrupt API. The enhanced API maintains compatibility with the IIC. The legacy HAL interrupt API is deprecated. Upgrading your device driver is very simple, requiring only minor changes to some function calls.

Table 27. HAL Interrupt Legacy and Enhanced API Functions

Legacy API Function	Enhanced API Function
alt_irq_register()	alt_ic_isr_register()
alt_irq_disable()	alt_ic_irq_disable()
alt_irq_enable()	alt_ic_irq_enable()

Related Information

- [Nios V Processor Software Developer Handbook](#)
For more information about exception handling.
- [Nios V Processor Reference Manual](#)
For more information about interrupt controller.

6.3. Shadow Register

Both the Nios II and Nios V processors support shadow registers, which were developed solely to improve context switching upon an interrupt. Switching between the general-purpose register (GPR) and the shadow register can complete context switching faster than the usual method of storing and restoring the GPR. However, the implementation of the shadow register is different in both processors.

Table 28. Comparison between Processors

Traits of Shadow Register	Nios V Processor	Nios II Processor
Number of supported register sets	Depended on selected interrupt controller	Up to 63 shadow register sets
Shadowed register coverage	• Only the upper 16 of the Nios V processor GPRs • All 32 floating-point registers of the Nios V processor floating point unit.	All 32 Nios II processor GPRs
Save and restore	Only the lower 16 of the Nios V processor GPRs	Not required
Associated processor CSR	• msrfstatus register • mrdpsrf register • mwrpsrf register	status register
continued...		

Traits of Shadow Register	Nios V Processor	Nios II Processor
Associated processor instructions	None	- rdprs - wrprs
Copy data between register sets	Executed by performing CSRRW instruction on either mrdpsrf or mwrpsrf CSR.	Executed by performing either rdprs or wrprs instruction.
Initialization with shadow register sets	Assert msrfstatus.ESI bit field as logic 1.	Copy the following registers from GPR to all shadow register sets: - Global pointer - Stack pointer - zero

6.4. Ethernet Stack

Nios V processor supports MicroC/TCP-IP protocol stack and it is not compatible with the retired Nios II Nichestack Ethernet Stack. You may use Altera-provided uC/TCP-IP design examples as baseline and port over other peripherals.

Note: Altera recommends re-instantiating the Triple Speed Ethernet IP and DMA Controller IP when upgrading from the Nios II Processor to the Nios V Processor.

Table 29. Board Specific Configuration

Soft IPs	Nios V Processor	Nios II Processor
DMA Controller	- Only support mSGDMA for both Quartus Prime Standard Edition and Quartus Prime Pro Edition - Enable extended feature support	- Quartus Prime Standard Edition supports - SGDMA - mSGDMA - Quartus Prime Pro Edition supports mSGDMA only - Disable extended feature support

Table 30. BSP Settings

BSP Settings	Nios II Processor	Nios V Processor
cflag_defined_symbols	-DTSE_MY_SYSTEM	-DTSE_MY_SYSTEM -DALT_DEBUG
cflag_warning	-Wall	-Wall -Wformat -Wformat-security
enable_cflag_fstack_protector_strong	-	Enable
enable_cflag_wformat_security	-	Enable
Enable JTAG UART small driver	Disable	Enable
DHCP Client	Enable in BSP Editor	Enable in application
Static IP Address	- Disable DHCP Client in BSP Editor - Set static IP address in application	- Disable DHCP Client in application - Set static IP address in application
Software package	altera_iniche	uc_tcp_ip
ucosii.os_tmr_en	Disable	Enable
ucosii.os_max_event	60	80

Related Information

- [Nios V Embedded Processor Design Handbook](#)
Nios® V Processor - Using the MicroC/TCP-IP Stack
- [Nios V Processor Software Developer Handbook](#)
For more information about MicroC/TCP-IP Protocol Stack

6.5. Bootloader

Both Nios II and Nios V processors support:

- Execute-in-place (XIP) using `alt_load` function
- Program copied to system RAM using boot copier

Table 31. Supported Flash Memories with Respective Boot Options

Supported Boot Memories	Device	Nios V Processor Booting Methods	Application Runtime Location	Boot Copier
On-Chip Flash (for Internal configuration)	MAX® 10 FPGA devices only (with On-Chip Flash IP)	Nios V processor application execute-in-place from On-Chip Flash	On-Chip Flash (XIP) + OCRAM/ External RAM (for writable data sections)	alt_load() function
		Nios V processor application copied from On-Chip Flash to RAM using boot copier	OCRAM/ External RAM	Reusing Bootloader via GSFI
General Purpose QSPI Flash (for user data only)	All supported FPGA devices (with Generic Serial Flash Interface FPGA IP)	Nios V processor application execute-in-place from general purpose QSPI flash	General purpose QSPI flash (XIP) + OCRAM/ External RAM (for writable data sections)	alt_load() function
		Nios V processor application copied from general purpose QSPI flash to RAM using boot copier	OCRAM/ External RAM	Bootloader via GSFI
Configuration QSPI Flash (for Active Serial configuration)	Control block-based devices (with Generic Serial Flash Interface Altera FPGA IP) ⁽²⁾	Nios V processor application execute-in-place from configuration QSPI flash.	Configuration QSPI flash (XIP) + OCRAM/ External RAM (for writable data sections)	alt_load() function
		Nios V processor application copied from configuration QSPI flash to RAM using boot copier.	OCRAM/ External RAM	GSFI bootloader
continued...				

⁽²⁾ Refer to *AN 980: Nios V Processor Quartus Prime Software Support* for the device list.

Supported Boot Memories	Device	Nios V Processor Booting Methods	Application Runtime Location	Boot Copier
	SDM-based devices (with Mailbox Client Altera FPGA IP). ⁽²⁾	Nios V processor application copied from configuration QSPI flash to RAM using boot copier.	OCRAM/ External RAM	SDM bootloader
On-chip Memory (OCRAM)	All supported Altera FPGA devices. ⁽²⁾	Nios V processor application execute-in-place from OCRAM.	OCRAM	alt_load() function
Tightly Coupled Memory (TCM)	All supported Altera FPGA devices. ⁽²⁾	Nios V processor application execute-in-place from TCM.	Instruction TCM (XIP) + Data TCM (for writable data sections)	None

6.6. Data and Instruction Cache

Nios II/f and Nios V/g processors support cache.

Table 32. Cache Comparison Between Nios V and Nios II Processors

Cache	Nios V/g Processor	Nios II/f Processor
Instruction Cache	- No Cache 1KB 2KB 4KB 8KB 16KB	0 512B 1KB 2KB 4KB 8KB 16KB 32KB 64KB
Data Cache	- No Cache 1KB 2KB 4KB 8KB 16KB	0 512B 1KB 2KB 4KB 8KB 16KB 32KB 64KB
Cache Bypass Method	Peripheral Regions	- Bit-31 cache bypass - Cache bypass instructions - Peripheral Regions

The processors use the peripheral regions to define non-cacheable address space for peripherals such as UART, PIO, DMA, and others. Both Nios II/f and Nios V/g processors support peripheral regions. Nios V/g processor supports two separate peripheral regions, and Nios II/f processor supports only one.

- For Nios II/f processor (with caches enabled), it is optional to use a peripheral region when using peripherals.
- For Nios V/g processor, you must place your peripherals in a peripheral region. Failure to do so can cause the processor system to malfunction.

If you do not define the peripherals regions, Platform Designer prompts the following warning message:

No peripheral regions have been defined. If your design contains any peripherals such as UART, PIO, DMA, etc, they must be placed within a peripheral region.

6.7. Tightly Coupled Memory

Nios II/f and Nios V/g processors support tightly coupled memory (TCM).

Table 33. TCM Characteristics

TCM	Nios V/g Processor	Nios II/f Processor
Number of Instruction TCM (ITCM)	Two	Four
Number of Data TCM (DTCM)	Two	Four
Memory Type	Dual-port (Internal port to processor and external subordinate port).	Depends on the connected on-chip memory: - Single-port - Dual-port
TCMs Size	0 Bytes (Disabled), up to 512 MBytes	Depends on the connected on-chip memory.
TCMs Location	Internal within Nios V processor core (internal RAM).	External to Nios II processor core (system RAM).
TCMs Instantiation	Instantiated within the processor upon enable.	- Expose TCM memory interface master port upon enable. - A connection must be made to any on-chip memory.
Memory Initialization	Supports HEX or MIF file.	Supports HEX or MIF file.

Table 34. TCM Use Cases

TCM	Nios V/g Processor	Nios II/f Processor
Memory Region	- Dual-port ITCM: - Instruction memory region (Internal connection to processor instruction bus) - Data memory region (Connects ITCM subordinate port to Nios V processor data_manager port) - Single-port DTCM: Data memory region (Internal connection to processor data bus).	- Single-port ITCM: Instruction memory region (Connects to ITCM master port) - Dual-port ITCM: - Instruction memory region (Connects 1st slave port to ITCM master port) - Data memory region (Connects 2nd slave port to Nios II processor data_master port) - Single-port DTCM: Data memory region (Connects to DTCM master port)
Nios V Processor Application Execute-In-Place from ITCM	Initialize dual-port ITCM and single-port DTCM (or system RAM) with Application HEX file.	- Initialize single-port ITCM and DTCM (or system RAM) with Application HEX file. - Initialize dual-port ITCM with Application HEX file.

continued...

TCM	Nios V/g Processor	Nios II/f Processor
alt_load() with ITCM	Supported with dual-port ITCM and single-port DTCM.	- Not supported with single-port ITCM. - Supported with dual-port ITCM.
Download Nios V Processor Application into ITCM	Supported with dual-port ITCM and single-port DTCM.	- Not supported with single-port ITCM. - Supported with dual-port ITCM.
External Host Interface	- Accessible by external host or processor core (for dual-port TCMs). - The interface is AXI4-Lite Subordinate Port.	- Accessible by external host or processor core (for dual-port TCMs). - The interface is Avalon Memory-Mapped Slave Port.

Figure 47. Tightly-Coupled Memory in Nios V/g Processor

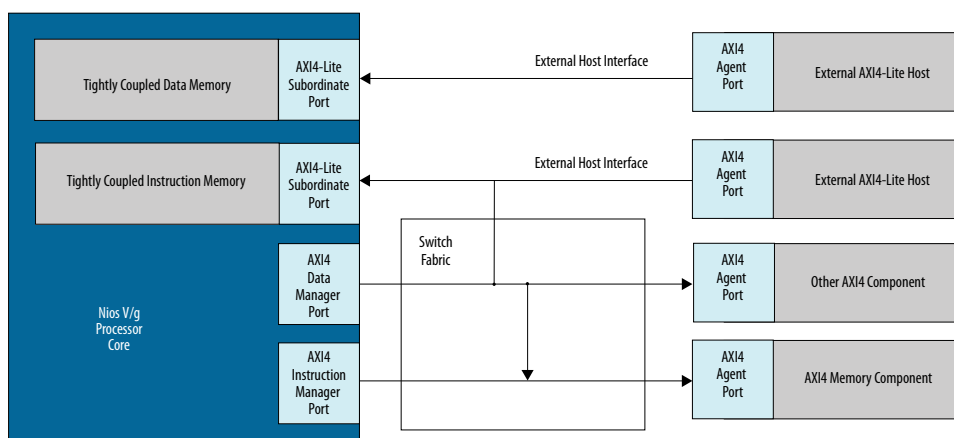
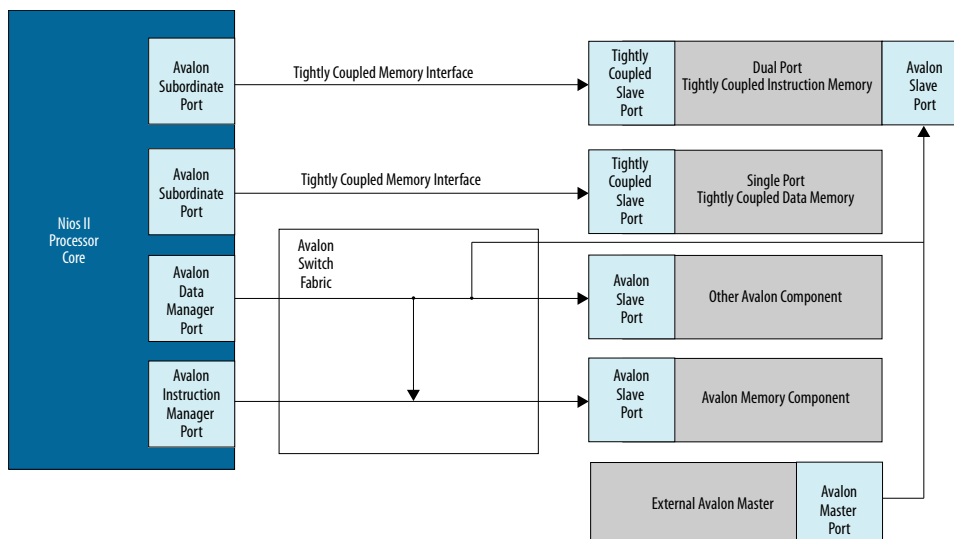


Figure 48. Tightly-Coupled Memory with Nios II/f Processor



6.8. Custom Instructions

Nios V/g and Nios II/f processors support custom instruction.

Table 35. Custom Instructions Comparison

Feature	Nios V/g Processor	Nios II/f Processor
Supported custom instruction types	• Multicycle • Extended • External Interface	• Combinational • Multicycle • Extended • Internal Register Files • External Interface
Combinational instructions	Implemented in two cycles.	Implemented in a single cycle.
Multicycle instructions	Supports variable length multicycle instructions.	Supports fixed length and variable length multicycle instructions.
Extended instructions	Apply the signal <code>ctrl[31:0]</code> to select the appropriate instruction.	Apply the signal <code>n[7:0]</code> to select the appropriate instruction.
Internal Register Files instructions	Not Supported	Supported
Manager Interface	Nios V Processor Custom Instruction Manager	Nios II Processor Custom Instruction Manager
Agent Interface	Nios V Processor Custom Instruction Subordinate	Nios II Processor Custom Instruction Subordinate
Signals and Interface	• <code>clk</code> • <code>reset</code> • <code>data0</code> • <code>data1</code> • <code>enable</code> • <code>result</code> • <code>done</code> • <code>alu_result</code>⁽³⁾ • <code>ctrl</code>	• <code>clk</code> • <code>reset</code> • <code>dataa</code> • <code>datab</code> • <code>start</code> • <code>result</code> • <code>done</code> • <code>clk_en</code>⁽⁴⁾
Built-in functions (To use non-integer types with instructions)	Not supported	Supported
FPH2 Hardware	Replaced with internal FPU	Supported

6.9. Error Correction Code

All Nios V processor variant and Nios II/f processors support ECC (except for Nios II/e processor).

⁽³⁾ `alu_result` is not equivalent to `clk_en`.

⁽⁴⁾ Nios V Processor Custom Instruction does not support `clk_en` signal.

Table 36. ECC Comparison

ECC	Nios V Processor	Nios II/f Processor
Supported variant	- Nios V/m - Nios V/g - Nios V/c	Nios II/f
ECC Detection	1-bit ECC error 2-bit ECC error	1-bit ECC error 2-bit ECC error
ECC Recovery	Available for instruction cache, or any 1-bit ECC errors in Nios V/g processor only.	Available for instruction cache, or any 1-bit ECC errors.
Output Signals	A conduit group, called <code>cpu_ecc_status</code> .	An Avalon streaming source, called <code>ecc_event_bus</code> .
Status Reporting	<code>cpu_ecc_status</code> indicates the error status using 2-bit label encoding. <code>cpu_ecc_source</code> indicates the error source using 4-bit label encoding.	29-bit one-hot encoding representing: - Affected internal RAM blocks, and - Error type.

6.10. Altera HAL Settings

The Nios II processor embedded peripherals HAL driver is backward compatible with the Nios V processor with the following `_sw.tcl` settings changes.

Table 37. Changes in the `_sw.tcl` Settings for the Nios V Processor

Nios V Processor (hal.*)	Nios II Processor (hal.*)
<code>enable_instruction_related_exceptions_api</code>	<code>enable_instruction_related_exceptions_api</code>
<code>max_file_descriptors</code>	<code>max_file_descriptors</code>
—	<code>disable_startup_thread_sync</code>
—	<code>enable_small_stack</code>
—	<code>exclude_default_exception</code>
<code>sys_clk_timer</code>	<code>sys_clk_timer</code>
<code>timestamp_timer</code>	<code>timestamp_timer</code>
<code>linker.allow_code_at_reset</code>	<code>linker.allow_code_at_reset</code>
<code>linker.enable_alt_load</code>	<code>linker.enable_alt_load</code>
<code>linker.enable_alt_load_copy_exceptions</code>	<code>linker.enable_alt_load_copy_exceptions</code>
<code>linker.enable_alt_load_copy_rodata</code>	<code>linker.enable_alt_load_copy_rodata</code>
<code>linker.enable_alt_load_copy_rwdata</code>	<code>linker.enable_alt_load_copy_rwdata</code>
<code>linker.enable_exception_stack</code>	<code>linker.enable_exception_stack</code>
<code>linker.exception_stack_memory_region_name</code>	<code>linker.exception_stack_memory_region_name</code>
<code>linker.exception_stack_size</code>	<code>linker.exception_stack_size</code>
—	<code>linker.enable_interrupt_stack</code>
—	<code>linker.interrupt_stack_memory_region_name</code>
—	<code>linker.interrupt_stack_size</code>
<code>linker.use_picolibc</code>	—
<i>continued...</i>	

Nios V Processor (hal. *)	Nios II Processor (hal. *)
toolchain.prefix	—
toolchain.enable_executable_overrides	—
toolchain.ar	make.ar
toolchain.as	make.as
toolchain.cc	make.cc
toolchain.cxx	make.cxx
toolchain.objdump	—
—	make: • build_pre_process • build_post_process • ar_pre_process • ar_post_process • as_pre_process • as_post_process • cc_pre_process • cc_post_process • cxx_pre_process • cxx_post_process
—	make.bsp_arflags
make.asflags	make.bsp_asflags
make.cflags_debug	make.bsp_cflags_debug
make.cflags_defined_symbols	make.bsp_cflags_defined_symbols
make.cflags_optimization	make.bsp_cflags_optimization
make.cflags_undefined_symbols	make.bsp_cflags_undefined_symbols
make.cflags_user_flags	make.bsp_cflags_user_flags
make.cflags_warnings	make.bsp_cflags_warnings
make.cxx_flags	make.bsp_cxx_flags
—	make.bsp_inc_dirs
make.link_flags	—
make.objdump_flags	—
make.enable_cflag_fstack_protector_strong	make.enable_cflag_fstack_protector_strong
make.enable_cflag_wformat_security	make.enable_cflag_wformat_security
—	make.ignore_system_derived: • big_endian • fpu_present • hardware_divide_present • hardware_fp_cust_inst_divider_present • hardware_fp_cust_inst_no_divider_present • soc_simulation_enabled • debug_core_present • hardware_multiplier_present • hardware_mulx_present
continued...	

Nios V Processor (hal.*)	Nios II Processor (hal.*)
	- sopc_system_base_address - sopc_system_id - sopc_system_timestamp
—	make.rm
—	custom_newlib_flags
enable_c_puls_plus	enable_c_plus_plus
enable_clean_exit	enable_clean_exit
enable_exit	enable_exit
—	enable_gprof
enable_reduced_device_drivers	enable_reduced_device_drivers
enable_lightweight_device_driver_api	enable_lightweight_device_driver_api
—	enable_mul_div_emulation
enable_runtime_stack_checking	enable_runtime_stack_checking
enable_sim_optimize	enable_sim_optimize
—	enable_small_c_library
—	enable_sopc_sysid_check
log_port	log_port
log_flags	log_flags
stderr	stderr
stdin	stdin
stdout	stdout
—	thread_stack_size

6.11. Micrium MicroC/OS-II BSP Settings

Nios II MicroC/OS-II BSP settings is backward compatible with Nios V processor.

Table 38. Changes in the MicroC/OS-II BSP Settings for Nios V Processor

Nios V Processor (ucosii.*)	Nios II Processor (ucosii.*)
os_thread_safe_c_lib	os_thread_safe_newlib
os_max_tasks	os_max_tasks
os_lower_prio	os_lower_prio
os_flag_en	os_flag_en
os_mutex_en	os_mutex_en
os_sem_en	os_sem_en
os_mbox_en	os_mbox_en
os_q_en	os_q_en
os_mem_en	os_mem_en
<i>continued...</i>	

Nios V Processor (ucosii.*)	Nios II Processor (ucosii.*)
os_tmr_en	os_tmr_en
os_arg_chk_en	os_arg_chk_en
os_cpu_hooks_en	os_cpu_hooks_en
os_debug_en	os_debug_en
os_sched_lock_en	os_sched_lock_en
os_task_stat_en	os_task_stat_en
os_task_reg_tbl_size	-
os_task_stat_stk_chk_en	os_task_stat_stk_chk_en
os_tick_step_en	os_tick_step_en
-	os_event_name_size
os_event_name_en	-
os_max_events	os_max_events
os_task_idle_stk_size	os_task_idle_stk_size
os_task_stat_stk_size	os_task_stat_stk_size
os_task_change_prio_en	os_task_change_prio_en
os_task_create_en	os_task_create_en
os_task_create_ext_en	os_task_create_ext_en
os_task_del_en	os_task_del_en
-	os_task_name_size
os_task_name_en	-
os_task_profile_en	os_task_profile_en
os_task_query_en	os_task_query_en
os_task_suspend_en	os_task_suspend_en
os_task_sw_hook_en	os_task_sw_hook_en
os_flag_wait_clr_en	os_flag_wait_clr_en
os_flag_accept_en	os_flag_accept_en
os_flag_del_en	os_flag_del_en
os_flag_query_en	os_flag_query_en
-	os_flag_name_size
os_flag_name_en	-
os_flags_nbits	os_flags_nbits
os_max_flags	os_max_flags
os_mutex_accept_en	os_mutex_accept_en
os_mutex_del_en	os_mutex_del_en
os_mutex_query_en	os_mutex_query_en
os_sem_accept_en	os_sem_accept_en
continued...	

Nios V Processor (ucosii.*)	Nios II Processor (ucosii.*)
os_sem_set_en	os_sem_set_en
os_sem_del_en	os_sem_del_en
os_sem_query_en	os_sem_query_en
os_mbox_accept_en	os_mbox_accept_en
os_mbox_del_en	os_mbox_del_en
os_mbox_post_en	os_mbox_post_en
os_mbox_post_opt_en	os_mbox_post_opt_en
os_mbox_query_en	os_mbox_query_en
os_q_accept_en	os_q_accept_en
os_q_del_en	os_q_del_en
os_q_flush_en	os_q_flush_en
os_q_post_en	os_q_post_en
os_q_post_front_en	os_q_post_front_en
os_q_post_opt_en	os_q_post_opt_en
os_q_query_en	os_q_query_en
os_max_qs	os_max_qs
os_mem_query_en	os_mem_query_en
-	os_mem_name_size
os_mem_name_en	-
os_max_mem_part	os_max_mem_part
os_task_tmr_stk_size	os_task_tmr_stk_size
os_task_tmr_prio	os_task_tmr_prio
os_tmr_cfg_max	os_tmr_cfg_max
-	os_tmr_cfg_name_size
os_tmr_cfg_name_en	-
os_tmr_cfg_ticks_per_sec	os_tmr_cfg_ticks_per_sec
os_tmr_cfg_wheel_size	os_tmr_cfg_wheel_size

6.12. Software Packages

The following table shows a list of supported and unsupported software packages by Nios V processor.

Table 39. Software Packages

Supported	Not Supported
- altera_ro_zipfs - uc_tcp_ip	- altera_hostfs - altera_iniche

7. Assembly Language Comparison

The following pages list all Nios II processor instruction mnemonics, along with the Nios V processor counterparts. To achieve similar functionality to a Nios II processor instruction, you can use either a supported Nios V processor instruction or a combination of instructions.

Table 40. Notation Conventions

Notation	Meaning
$X \leftarrow Y$	X is written with Y
$PC \leftarrow X$	The program counter (PC) is written with address X; the instruction at X is the next instruction to execute
PC	The address of the assembly instruction in question
rA, rB, rC	One of the 32-bit general-purpose registers in Nios II processor
xA, xB, xC	One of the 32-bit general-purpose registers in Nios V processor
prs.rA	General-purpose register rA in the previous register set
IMMn	An n-bit immediate value, embedded in the instruction word
IMMED	An immediate value
X_n	The nth bit of X, where n = 0 is the LSB
$X_{n..m}$	Consecutive bits n through m of X
0xNNMM	Hexadecimal notation
$X : Y$	Bitwise concatenation For example, $(0x12 : 0x34) = 0x1234$
$\sigma(X)$	The value of X after being sign-extended to a full register-sized signed integer
$X \gg n$	The value X after being right-shifted n bit positions
$X \ll n$	The value X after being left-shifted n bit positions
$X \& Y$	Bitwise logical AND
$X Y$	Bitwise logical OR
$X \wedge Y$	Bitwise logical XOR
$\sim X$	Bitwise logical NOT (one's complement)
Mem8[X]	The byte located in data memory at byte address X
Mem16[X]	The halfword located in data memory at byte address X
Mem32[X]	The word located in data memory at byte address X
label	An address label specified in the assembly file
(signed) rX	The value of rX treated as a signed number
(unsigned) rX	The value of rX treated as an unsigned number

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera and Intel warrant performance of its FPGA and semiconductor products to current specifications in accordance with Altera's or Intel's standard warranty as applicable, but reserves the right to make changes to any products and services at any time without notice. Altera and Intel assume no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera or Intel. Altera and Intel customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.

*Other names and brands may be claimed as the property of others.

Related Information

- [Nios® II Processor Reference Guide: Instruction Set Reference](#)
- [Nios V Processor Reference Manual: Nios® V/c Processor Instruction Set Reference](#)
- [Nios V Processor Reference Manual: Nios® V/m Processor Instruction Set Reference](#)
- [Nios V Processor Reference Manual: Nios® V/g Processor Instruction Set Reference](#)

7.1. Arithmetic and Logic Operations

Table 41. Assembly Language Comparison (Arithmetic and Logic Operations)

Nios II Processor		Nios V Processor	
Operation	Assembler Syntax	Operation	Assembler Syntax
$rC \leftarrow rA + rB$	add rC, rA, rB	$xC \leftarrow xA + xB$	add xC, xA, xB
$rB \leftarrow rA + \sigma(\text{IMM16})$	addi rB, rA, IMM16	$xB \leftarrow xA + \sigma(\text{IMM12})$	addi xB, xA, IMM12
$rC \leftarrow rA \& rB$	and rC, rA, rB	$xC \leftarrow xA \& xB$	and xC, xA, xB
$rB \leftarrow rA \& (\text{IMM16} : 0x0000)$	andhi rB, rA, IMM16	$xB \leftarrow xA \& (\text{IMM20} : 0x000)$	lui xB, IMM20 and xB, xA, xB
$rB \leftarrow rA \& (0x0000 : \text{IMM16})$	andi rB, rA, IMM16	$xB \leftarrow xA \& \sigma(\text{IMM12})$	andi xB, xA, IMM12
$rB \leftarrow rA \& (0x0000 : \text{IMM16})$	andi rB, rA, IMM16	$xB \leftarrow xA \& (0x0000 : \text{IMM16})$	li xB, IMM16 and xB, xA, xB
if (rA == rB) then rC ← 1 else rC ← 0	cmpeq rC, rA, rB	if (xA - xB == 0) then xC ← 1 else xC ← 0	sub xB, xA, xB seqz xC, xB
if (rA == $\sigma(\text{IMM16})$) then rB ← 1 else rB ← 0	cmpeqi rB, rA, IMM16	if (xA - $\pm\text{IMM15}$ == 0) then xB ← 1 else xB ← 0	li xB, $\pm\text{IMM15}$ sub xB, xA, xB seqz xB, xB
if ((signed) rA >= (signed) rB) then rC ← 1 else rC ← 0	cmpge rC, rA, rB	if ((signed) xA < (signed) xB) then xC ← 0 else xC ← 1	slt xC, xA, xB not xC, xC andi xC, xC, 1
if ((signed) rA >= (signed) $\sigma(\text{IMM16})$) then rB ← 1 else rB ← 0	cmpgei rB, rA, IMM16	if ((signed) xA < (signed) $\sigma(\text{IMM12})$) then xB ← 0 else xB ← 1	slti xB, xA, IMM12 not xB, xB andi xB, xB, 1
if ((unsigned) rA >= (unsigned) rB) then rC ← 1 else rC ← 0	cmpgeu rC, rA, rB	if ((unsigned) xA < (unsigned) xB) then xC ← 0 else xC ← 1	sltu xC, xA, xB not xC, xC andi xC, xC, 1
if ((unsigned) rA >= (unsigned) $(0x0000 : \text{IMM16})$) then rB ← 1 else rB ← 0	cmpgeui rB, rA, IMM16	if ((unsigned) xA < (unsigned) $(0x00000 : \text{IMM12})$) then xB ← 0 else xB ← 1	sltiu xB, xA, IMM12 not xB, xB andi xB, xB, 1
if ((signed) rA > (signed) rB)	cmpgt rC, rA, rB	if ((signed) xA > (signed) xB)	slt xC, xB, xA

continued...

Nios II Processor		Nios V Processor	
Operation	Assembler Syntax	Operation	Assembler Syntax
then rC ← 1 else rC ← 0		then xC ← 1 else xC ← 0	
if ((signed) rA > (signed) σ(IMM16)) then rB ← 1 else rB ← 0	cmpgti rB, rA, IMM16	if ((signed) xA > (signed) IMM15) then xB ← 1 else xB ← 0	li xB, ±IMM15 slt xB, xB, xA
if ((unsigned) rA > (unsigned) rB) then rC ← 1 else rC ← 0	cmpgtu rC, rA, rB	if ((unsigned) xA > (unsigned) xB) then xC ← 1 else xC ← 0	sltu xC, xB, xA
if ((unsigned) rA > (unsigned) (0x0000 : IMM16)) then rB ← 1 else rB ← 0	cmpgtui rB, rA, IMM16	if ((unsigned) xA > (unsigned) (0x0000 : IMM16)) then xB ← 1 else xB ← 0	li xB, IMM16 sltu xB, xB, xA
if ((signed) rA ≤ (signed) rB) then rC ← 1 else rC ← 0	cmple rC, rA, rB	if ((signed) xA > (signed) xB) then xC ← 0 else xC ← 1	slt xC, xB, xA not xC, xC andi xC, xC, 1
if ((signed) rA ≤ (signed) σ(IMM16)) then rB ← 1 else rB ← 0	cmplei rB, rA, IMM16	if ((signed) xA < (signed) σ(IMM12+1)) then xB ← 1 else xB ← 0	slti xB, xA, IMM12+1
if ((unsigned) rA ≤ (unsigned) rB) then rC ← 1 else rC ← 0	cmpleu rC, rA, rB	if ((unsigned) xB < (unsigned) xA) then xC ← 0 else xC ← 1	sltu xC, xB, xA not xC, xC andi xC, xC, 1
if ((unsigned) rA ≤ (unsigned) (0x0000 : IMM16)) then rB ← 1 else rB ← 0	cmpleui rB, rA, IMM16	if ((unsigned) xA < (unsigned) (0x00000 : IMM12+1)) then xB ← 1 else xB ← 0	sltiu xB, xA, IMM12+1
if ((signed) rA < (signed) rB) then rC ← 1 else rC ← 0	cmplt rC, rA, rB	if ((signed) xA < (signed) xB) then xC ← 1 else xC ← 0	slt xC, xA, xB
if ((signed) rA < (signed) σ(IMM16)) then rB ← 1 else rB ← 0	cmplti rB, rA, IMM16	if ((signed) xA < (signed) σ(IMM12)) then xC ← 1 else xC ← 0	slti xC, xA, IMM12
if ((unsigned) rA < (unsigned) rB) then rC ← 1 else rC ← 0	cmpltu rC, rA, rB	if ((unsigned) xA < (unsigned) xB) then xC ← 1 else xC ← 0	sltu xC, xA, xB
if ((unsigned) rA < (unsigned) (0x0000 : IMM16)) then rB ← 1 else rB ← 0	cmpltui rB, rA, IMM16	if ((unsigned) xA < (unsigned) (0x00000 : IMM12)) then xC ← 1 else xC ← 0	sltiu xC, xA, IMM12
if (rA != rB) then rC ← 1 else rC ← 0	cmpne rC, rA, rB	If (xA - xB != 0) then xC ← 1 else xC ← 0	sub xC, xA, xB snez xC, xC
if (rA != σ(IMM16)) then rB ← 1	cmpnei rB, rA, IMM16	If (xA - (signed)IMM15 != 0) then xC ← 1	li xB, ±IMM15
continued...			

Nios II Processor		Nios V Processor	
Operation	Assembler Syntax	Operation	Assembler Syntax
else $rB \leftarrow 0$		else $xC \leftarrow 0$	sub xC, xA, xB snez xC, xC
$rC \leftarrow (\text{signed}) rA \div (\text{signed}) rB$	div rC, rA, rB	$xC \leftarrow (\text{signed}) xA \div (\text{signed}) xB$	div xC, xA, xB
$rC \leftarrow (\text{unsigned}) rA \div (\text{unsigned}) rB$	divu rC, rA, rB	$xC \leftarrow (\text{unsigned}) xA \div (\text{unsigned}) xB$	divu xC, xA, xB
$rC \leftarrow (rA \times rB)_{31..0}$	mul rC, rA, rB	$xC \leftarrow (xA \times xB)_{31..0}$	mul xC, xA, xB
$rB \leftarrow (rA \times \sigma(\text{IMM16}))_{31..0}$	muli $rB, rA, \text{IMM16}$	$xB \leftarrow \sigma(\text{IMM12})$ $xC \leftarrow (xA \times xB)_{31..0}$	addi $xB, x0, \text{IMM12}$ mul xB, xA, xB
$rC \leftarrow ((\text{signed}) rA) \times ((\text{signed}) rB)_{63..32}$	mulxss rC, rA, rB	$xC \leftarrow ((\text{signed}) xA) \times ((\text{signed}) xB)_{63..32}$	mulh xC, xA, xB
$rC \leftarrow ((\text{signed}) rA) \times ((\text{unsigned}) rB)_{63..32}$	mulxsu rC, rA, rB	$xC \leftarrow ((\text{signed}) xA) \times ((\text{unsigned}) xB)_{63..32}$	mulhsu xC, xA, xB
$rC \leftarrow ((\text{unsigned}) rA) \times ((\text{unsigned}) rB)_{63..32}$	mulxuu rC, rA, rB	$xC \leftarrow ((\text{unsigned}) xA) \times ((\text{unsigned}) xB)_{63..32}$	mulhu xC, xA, xB
$rC \leftarrow \sim(rA \mid rB)$	nor rC, rA, rB	$xC \leftarrow xA \mid xB$ $xC \leftarrow \sim xC$	or xC, xA, xB not xC, xC
$rC \leftarrow rA \mid rB$	or rC, rA, rB	$xC \leftarrow xA \mid xB$	or xC, xA, xB
$rB \leftarrow rA \mid (\text{IMM16} : 0x0000)$	orhi $rB, rA, \text{IMM16}$	$xB \leftarrow xA \mid (\text{IMM20} : 0x0000)$	lui $xC, \text{IMM20}$ or xB, xA, xC
$rB \leftarrow rA \mid (0x0000 : \text{IMM16})$	ori $rB, rA, \text{IMM16}$	$xB \leftarrow xA \mid \sigma(\text{IMM12})$	ori $xB, xA, \text{IMM12}$
$rB \leftarrow rA \mid (0x0000 : \text{IMM16})$	ori $rB, rA, \text{IMM16}$	$xC \leftarrow (0x0000 : \text{IMM16})$ $xB \leftarrow xA \mid xC$	li $xC, \text{IMM16}$ or xB, xA, xC
$rC \leftarrow rA$ rotated left $rB_{4..0}$ bit positions	rol rC, rA, rB	Not available in Nios V processor	-
$rC \leftarrow rA$ rotated left IMM5 bit positions	roli $rC, rA, \text{IMM5}$	Not available in Nios V processor	-
$rC \leftarrow rA$ rotated right $rB_{4..0}$ bit positions	ror rC, rA, rB	Not available in Nios V processor	-
$rC \leftarrow rA \ll (rB_{4..0})$	sll rC, rA, rB	$xC \leftarrow xA \ll (xB_{4..0})$	sll xC, xA, xB
$rC \leftarrow rA \ll \text{IMM5}$	slli $rC, rA, \text{IMM5}$	$xC \leftarrow xA \ll \text{IMM5}$	slli $xC, xA, \text{IMM5}$
$rC \leftarrow (\text{signed}) rA \gg ((\text{unsigned}) rB_{4..0})$	sra rC, rA, rB	$xC \leftarrow (\text{signed}) xA \gg ((\text{unsigned}) xB_{4..0})$	sra xC, xA, xB
$rC \leftarrow (\text{signed}) rA \gg ((\text{unsigned}) \text{IMM5})$	srai $rC, rA, \text{IMM5}$	$xC \leftarrow (\text{signed}) xA \gg ((\text{unsigned}) \text{IMM5})$	srai $xC, xA, \text{IMM5}$
$rC \leftarrow (\text{unsigned}) rA \gg ((\text{unsigned}) rB_{4..0})$	srl rC, rA, rB	$xC \leftarrow (\text{unsigned}) xA \gg ((\text{unsigned}) xB_{4..0})$	srl xC, xA, xB
$rC \leftarrow (\text{unsigned}) rA \gg ((\text{unsigned}) \text{IMM5})$	srl $rC, rA, \text{IMM5}$	$xC \leftarrow (\text{unsigned}) xA \gg ((\text{unsigned}) \text{IMM5})$	srl $xC, xA, \text{IMM5}$
$rC \leftarrow rA - rB$	sub rC, rA, rB	$xC \leftarrow xA - xB$	sub xC, xA, xB

continued...

Nios II Processor		Nios V Processor	
Operation	Assembler Syntax	Operation	Assembler Syntax
$rB \leftarrow rA - \sigma(\text{IMMED})$	subi rB, rA, IMMED	$xC \leftarrow xA - \sigma(\text{IMM12})$	addi xC, xA, -IMM12
$rC \leftarrow rA \wedge rB$	xor rC, rA, rB	$xC \leftarrow xA \wedge xB$	xor xC, xA, xB
$rB \leftarrow rA \wedge (\text{IMM16} : 0x0000)$	xorhi rB, rA, IMM16	$xB \leftarrow xA \wedge (\text{IMM20} : 0x000)$	lui xC, IMM20 xor xB, xA, xC
$rB \leftarrow rA \wedge (0x0000 : \text{IMM16})$	xori rB, rA, IMM16	$xB \leftarrow xA \wedge \sigma(\text{IMM12})$	xori xC, xA, IMM12
$rB \leftarrow rA \wedge (0x0000 : \text{IMM16})$	xori rB, rA, IMM16	$xC \leftarrow \text{IMM16}$ $xB \leftarrow xA \wedge xC$	li xC, IMM16 xor xB, xA, xC

7.2. Data Handling and Memory Operations

Table 42. Assembly Language Comparison (Data Handling and Memory Operations)

Nios II Processor		Nios V Processor	
Operation	Assembler Syntax	Operation	Assembler Syntax
Flushes the data cache line associated with address $rA + \sigma(\text{IMM16})$ based on line field.	flushd IMM16(rA)	Flush the data cache line associated with address $xA + \sigma(\text{IMM12})$ based on index field.	addi xA, xA, IMM12 lui xB, 0x000003e0 and xA, xA, xB srli xA, xA, 5 cbo.flush.ix xA
Refer to the detailed steps for the Nios V processor pseudo-instruction below: 1. Compute effective address by the sum of xA with offset IMM12. 2. Load based on cache size. In a 1 kilobytes cache, index bit field is 5-bit wide (0x3e0). 3. Extract the value of the 5-bit index from address $xA + \sigma(\text{IMM12})$ using AND operation. 4. Shift-right by 5 bits to remove offset bit field. 5. Flush the data cache line associated with the index of address $xA + \sigma(\text{IMM12})$.			
Flushes the data cache line currently caching address $rA + \sigma(\text{IMM16})$.	flushda IMM16(rA)	Flush the data cache line associated with address $xA + \sigma(\text{IMM12})$.	addi xA, xA, IMM12 cbo.flush 0(xA)
Flushes the instruction cache line associated with address rA.	flushi rA	To synchronize the instruction and data streams.	fence.i
Flushes the processor pipeline of any prefetched instructions.	flushp	To synchronize the instruction and data streams.	fence.i
Initializes the data cache line associated with address $rA + \sigma(\text{IMM16})$ based on line field.	initd IMM16(rA)	Initializes the data cache line associated with address $xA + \sigma(\text{IMM12})$ based on index field.	addi xA, xA, IMM12 lui xB, 0x000003fe0 and xA, xA, xB srli xA, xA, 5 cbo.inval.ix xA
Refer to the detailed steps for the Nios V processor pseudo-instruction below: 1. Compute effective address by the sum of xA with offset IMM12. 2. Load based on cache size. In a 16 kilobytes cache, index bit field is 9-bit wide (0x3fe0). 3. Extract the value of the 9-bit index from address $xA + \sigma(\text{IMM12})$ using AND operation 4. Shift-right by 5 bits to remove offset bit field. 5. Initializes the data cache line associated with the index of address $xA + \sigma(\text{IMM12})$.			
continued...			

Nios II Processor		Nios V Processor	
Operation	Assembler Syntax	Operation	Assembler Syntax
Initializes the data cache line currently caching address rA + $\sigma(\text{IMM16})$.	initda IMM16(rA)	Initializes the data cache line associated with address xA + $\sigma(\text{IMM12})$.	addi xA, xA, IMM12 cbo.inval 0(xA)
Initializes the instruction cache line associated with address rA.	initi rA	To synchronize the instruction and data streams.	fence.i
Load Byte from Memory	ldb rB, IMM16(rA)	Load Byte from Memory	lb xB, IMM12(xA)
Load Byte from I/O Peripheral.	ldbio rB, IMM16(rA)	Not Available in Nios V processor	-
Load Unsigned Byte from Memory	ldbu rB, IMM16(rA)	Load Unsigned Byte from Memory	lbu xB, IMM12(xA)
Load Unsigned Byte from I/O Peripheral	ldbuio rB, IMM16(rA)	Not Available in Nios V processor	-
Load Halfword from Memory	ldh rB, IMM16(rA)	Load Halfword from Memory	lh xB, IMM12(xA)
Load Halfword from I/O Peripheral	ldhio rB, IMM16(rA)	Not Available in Nios V processor	-
Load Unsigned Halfword from Memory	ldhu rB, IMM16(rA)	Load Unsigned Halfword from Memory	lhu xB, IMM12(xA)
Load Unsigned Halfword from I/O Peripheral	ldhuio rB, IMM16(rA)	Not Available in Nios V processor	-
Load 32-bit Word from Memory	ldw rB, IMM16(rA)	Load 32-bit Word from Memory	lw xB, IMM12(xA)
Load 32-bit Word from I/O Peripheral	ldwio rB, IMM16(rA)	Not Available in Nios V processor	-
rC ← rA	mov rC, rA	xC ← xA	mv xC, xA
rB ← (IMMED : 0x0000)	movhi rB, IMMED	xB ← (IMMED: 0x0000)	li xB, IMMED li xC, 0xffff0000 and xB, xB, xC
rB ← $\sigma(\text{IMMED})$	movi rB, IMMED	xB ← $\sigma(\text{IMM12})$	addi xB, x0, IMM12
Move Immediate Address into Word	movia rB, label	Move Immediate Address into Word	lla xB, label
rB ← IMMED	movui rB, IMMED	xB ← IMMED	li xB, IMMED
Store Byte to Memory	stb rB, IMM16(rA)	Store Byte to Memory	sb xB, IMM12(xA)
Store Byte to I/O Peripheral	stbio rB, IMM16(rA)	Not Available in Nios V processor	-
Store Halfword to Memory	sth rB, IMM16(rA)	Store Halfword to Memory	sh xB, IMM12(xA)
Store Halfword to I/O Peripheral	sthio rB, IMM16(rA)	Not Available in Nios V processor	-
Store 32-bit Word to Memory	stw rB, IMM16(rA)	Store 32-bit Word to Memory	sw xB, IMM12(xA)
Store 32-bit Word to I/O Peripheral	stwio rB, IMM16(rA)	Not Available in Nios V processor	-

Related Information

Nios V Processor Reference Manual: [Cache Memory](#)

7.3. Control Flow Operations

Table 43. Assembly Language Comparison (Control Flow Operations)

Nios II Processor		Nios V Processor	
Operation	Assembler Syntax	Operation	Assembler Syntax
if (rA == rB) then PC ← label else PC ← PC + 4	beq rA, rB, label	if (xA == xB) then PC ← label else PC ← PC + 4	beq xA, xB, label
if ((signed) rA >= (signed) rB) then PC ← label else PC ← PC + 4	bge rA, rB, label	if ((signed) xA >= (signed) xB) then PC ← label else PC ← PC + 4	bge xA, xB, label
if ((unsigned) rA >= (unsigned) rB) then PC ← label else PC ← PC + 4	bgeu rA, rB, label	if ((unsigned) xA >= (unsigned) xB) then PC ← label else PC ← PC + 4	bgeu xA, xB, label
if ((signed) rA > (signed) rB) then PC ← label else PC ← PC + 4	bgt rA, rB, label	if ((signed) xA > (signed) xB) then PC ← label else PC ← PC + 4	bgt xA, xB, label
if ((unsigned) rA > (unsigned) rB) then PC ← label else PC ← PC + 4	bgtu rA, rB, label	if ((unsigned) xA > (unsigned) xB) then PC ← label else PC ← PC + 4	bgtu xA, xB, label
if ((signed) rA <= (signed) rB) then PC ← label else PC ← PC + 4	ble rA, rB, label	if ((signed) xA <= (signed) xB) then PC ← label else PC ← PC + 4	ble xA, xB, label
if ((unsigned) rA <= (unsigned) rB) then PC ← label else PC ← PC + 4	bleu rA, rB, label	if ((unsigned) xA <= (unsigned) xB) then PC ← label else PC ← PC + 4	bleu xA, xB, label
if ((signed) rA < (signed) rB) then PC ← label else PC ← PC + 4	blt rA, rB, label	if ((signed) xA < (signed) xB) then PC ← label else PC ← PC + 4	blt xA, xB, label
if ((unsigned) rA < (unsigned) rB) then PC ← label else PC ← PC + 4	bltu rA, rB, label	if ((unsigned) xA < (unsigned) xB) then PC ← label else PC ← PC + 4	bltu xA, xB, label
if (rA != rB) then PC ← label else PC ← PC + 4	bne rA, rB, label	if (xA != xB) then PC ← label else PC ← PC + 4	bne xA, xB, label
PC ← label	br label	PC ← label	j label
ra ← PC + 4 PC ← label	call label	x1 ← PC + 4 PC ← label	call label
ra ← PC + 4 PC ← rA	callr rA	x1 ← PC + 4 PC ← xA	jalr xA
Computed jump	jmp rA	Computed jump	jr xA
Jump Immediate	jmp_i label	Jump Immediate (Within 20-bits offset)	j label
Jump Immediate	jmp_i label	Jump Far Away Immediate (Within 32-bits offset)	tail label
Memory Synchronization	sync	Memory Synchronization	fence

7.4. Miscellaneous Operations

Table 44. Assembly Language Comparison (Miscellaneous Operations)

Nios II Processor		Nios V Processor	
Operation	Assembler Syntax	Operation	Assembler Syntax
Debugger Breakpoint	break	Debugger Breakpoint	ebreak
Debugger Breakpoint Return	bret	Debugger Breakpoint Return	dret
Custom Instruction	custom N, rC, rA, rB	Custom Instruction	Refer to custom_instruction_debug.xml to write the machine code. For example, ".4byte 0xe7978b". <i>Note:</i> If you use custom instructions to generate the BSP, you can find the .xml file in the BSP Project folder.
status ← estatus PC ← ea	eret	PC ← mepc	mret
rC ← PC + 4	nextpc rC	xC ← PC + 4 PC ← PC + 4	jal xC, 4
No Operation	nop	No Operation	nop
Read from Control Register	rdctl rC, ctln	Read from Control Register	csrr xC, csr
Read from Previous Register Set	rdprs rB, rA, IMM16	Not available in Nios V processor	-
Return from Subroutine	ret	Return from Subroutine	ret
Trap	trap	Trap	ecall
Write to Control Register	wrctl ctln, rA	Write to Control Register	csrw csr, xC
Read to Previous Register Set	wrprs rC, rA	Not Available in Nios V processor	-

8. Document Revision History for the AN 978: Nios V Processor Migration Guidelines

Document Version	Changes
2025.05.22	- Updated table <i>Integrated Development Environment (IDE) and Software Ecosystem Comparison Table</i> to add <i>Ashling Visual Studio Code Extension for Altera FPGAs</i>. - Updated <i>Upgrading Nios II/f Processor to Nios V/g Processor</i> to add content about Enable Core Level Interrupt Controller, Interrupt Mode, and Shadow Register Files: - Updated <i>IP Parameter Configuration</i> - Update the figures for Nios V/g processor IP parameter editor. - Updated table <i>IP Parameter Editor Comparison</i>. - Updated <i>Interrupt</i>. - Added new topic <i>Shadow Register</i>.
2025.01.27	- Added new parameters in the topic: <i>IP Parameter Configuration</i> for Nios V/m and Nios V/g variants. - Updated the following figures to the latest interface: <i>Nios V/m Microcontroller Intel FPGA IP – IP Parameter Editor</i> <i>IP Parameter Editor Comparison</i> - Updated the IP Parameter Editor in the table: <i>IP Parameter Editor Comparison</i> for Nios V/m and Nios V/g variants. - Updated the Interrupt Controller to Vectore Interrupt Controller in the table <i>Interrupt Feature Comparison</i>. - Updated <i>Ethernet Stack</i> to add the following tables: <i>Board Specific Configuration</i> <i>BSP Settings</i> - Updated the table <i>Supported Flash Memories with Respective Boot Options</i> to add the following supported boot memories: - On-Chip Flash (for Internal configuration) - General Purpose QSPI Flash (for user data only) - Added new table <i>Changes in the MicroC/OS-II BSP Settings for Nios V Processor</i>.
2024.11.25	- Edited <i>Overview</i> to add more clarity. - Updated the <i>Branch Prediction</i> value in the Table: <i>Core Architecture Comparison to Nios II/f Processor</i> to Yes for Nios V/g processor. - Changed the title for topic <i>Processor Core Migration Mapping to Processor Core Upgrade Paths</i>. - Removed <i>Quartus Prime Software Release Version</i> from the Table: <i>Processor Core Migration</i>. - Updated <i>Hardware Component</i> for Quartus Prime Pro Edition to add a note about enabling <i>Avalon Interface</i>. - Edited the following topics in <i>Migrating Nios II/e Processor to Nios V/c Processor</i>, <i>Migrating Nios II Processor to Nios V/m Processor</i>, and <i>Migrating Nios IIf Processor to Nios V/g Processor</i>: - Updated topic <i>Primary Interface</i> for Nios V/c and Nios V/m processors to add a note about using Avalon memory-mapped interface for the Data Bus and Instruction Bus interfaces. - Updated topic <i>IP Parameter Configuration</i> to add more information about the <i>CPU Architecture</i> and updated new IP parameter figures. - Updated the <i>Branch Prediction</i> value for the Nios V/g processor in the Table: <i>IP Parameter Editor Comparison</i>.

continued...

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera and Intel warrant performance of its FPGA and semiconductor products to current specifications in accordance with Altera's or Intel's standard warranty as applicable, but reserves the right to make changes to any products and services at any time without notice. Altera and Intel assume no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera or Intel. Altera and Intel customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.

*Other names and brands may be claimed as the property of others.

ISO
9001:2015
Registered

Document Version	Changes
	- Updated the <i>ECC Recovery</i> value for Nios V processor in the Table: <i>Error Correction Code</i>. - Added <i>enable_runtime_stack_checking</i> to the Table: <i>Intel HAL Settings</i>. - Added new chapter: <i>Assembly Language Comparison</i>.
2024.05.15	- Added notes in <i>Software Component</i> to refer to <i>Intel HAL Settings</i> to migrate BSP settings. - Edited <i>IP Parameter Configuration</i>. - Updated the <i>IP Parameter Editor</i> figures. - Updated the <i>IP Parameter Editor Comparison</i> table. - Edited <i>Interrupt</i>, <i>Data and Instruction Cache</i>, and <i>Tightly Coupled Memory</i> topics.
2023.11.12	- Removed <i>Atomic Extn</i> from the following tables: <i>Core Architecture Comparison to Nios II/e Processor</i>. <i>Core Architecture Comparison to Nios II/f Processor</i>. - Edited the topic <i>Software Optimization</i> - Updated the <i>Program Size (KB)</i> values for Nios V processor. - Added a link to <i>Nios® V Processor Software Developer Handbook - Reducing Code Footprint in Embedded Systems</i>. - Updated the figure for the <i>IP Parameter Editor</i> for Nios V/c, Nios V/m, and Nios V/g processors. - Replaced table <i>Supported Nios V Processor Booting Use Cases</i> with the table <i>Supported Flash Memories with Respective Boot Options</i>. - Update the TCM in the table <i>TCM Use Cases</i> for Nios V/g processor. - Updated the figure <i>Tightly-Coupled Memory in Nios V/g Processor</i> and the note
2023.10.02	- Updated the following topics for new Nios V/c processor variant and new Nios V/g features: <i>Software Optimization</i> <i>Design Migration from Nios II Processor to Nios V Processor</i> for Quartus Prime Pro Edition and Quartus Prime Standard Edition <i>Migrating Nios II Processor to Nios V/m Processor</i> <i>Migrating Nios II/f Processor to Nios V/g Processor</i> <i>Timer</i> - Added the following new topics: <i>Migrating Nios II/e Processor to Nios V/c Processor</i> <i>Tightly Coupled Memory</i> <i>Error Correction Code</i>
2023.08.22	Updated the Memory Management Unit (MMU) value for Nios II/f Processor in <i>Core Architecture Comparison Table</i> .
2023.04.14	Initial release.